

UNIVERSITY OF SOUTHERN DENMARK

SEMESTERPROJECT: INTELLIGENT SOFTWARE
SYSTEMS
T630012401

Better Reads - The Book Database

Group 6:

Beáta Sachs
Florian Alexander Goetz
Natalia Monika Fudali
Natasja Nicolaya Bruland
Oskar Veng Engel
Vladyslav Berezhnyi

Supervisor:
Jukka Ruohonen

May 26, 2025



Abstract

Finding genuinely relevant literature has become more and more difficult due to the increasingly overwhelming amount of available content. This project, *Better Reads*, addresses this challenge by developing a web-based book discovery and recommendation system. The system uses a combination of user data, such as browsing history and favorites, and constant book features such as book tags, ratings, and popularity to deliver personalized suggestions. Using Django web application framework, a PostgreSQL database, and further open-source technologies, the application provides free-text search with spell correction, filtering, and sorting mechanisms. A recommendation engine based on cosine similarity and weighted scoring provides personalized results to enhance user satisfaction. While the current implementation functions as a prototype with promising results, future work could incorporate machine learning and large-scale data handling to further improve personalization, as well as major performance improvement strategies.

Contents

1	Introduction	1
2	Problem Analysis	2
2.1	Existing tools and services	2
2.2	Project Aim	2
2.3	Objectives	2
2.4	Problem Statement	3
3	Methodology	4
3.1	Work Methods	4
3.2	Project Organization	4
3.3	Development Technologies	4
4	Requirements	5
4.1	External Project Requirements	5
4.2	Project Requirements	5
5	Design	7
5.1	Graphical User Interface	7
5.2	Architecture	7
5.3	Database and Datasheet	15
5.4	Filtering and Sorting	15
5.5	Text-based Search	16
5.6	Book Page visiting History	16
5.7	Book Recommendation	16
5.7.1	General Algorithm	16
5.7.2	Book Page	18
5.7.3	Search Page	19
5.7.4	Account Page	19
5.7.5	Afterthoughts on Precomputation	19
5.7.6	Afterthoughts on Machine Learning	20
6	Implementation	21
6.1	Technologies	21
6.2	Authentication	21
6.3	Database	22
6.4	Package Bundler (Webpack)	23
6.5	Search Functionality	23
6.5.1	Spellcheck functionality	24
6.6	Filtering and Sorting	25
6.7	Browsing History	25
6.8	Favorites	26

6.9	Recommendation	26
6.9.1	Book Page	26
6.9.2	Account and Search Page	28
7	Verification and Testing	30
7.1	Unit Testing	30
7.2	Integration Testing	31
7.3	Performance and Reliability Testing	32
7.4	Acceptance Testing	33
8	Conclusion	35
8.1	Summary	35
8.2	Future Work	36
A	User Interface Design	41
B	Project Proposal	45
C	C4 Model Diagrams	56
D	Conceptual Artificial Intelligence (AI) integration	59
E	Test Scripts	61
F	Meeting Logs	79

1 Introduction

In a digital era defined by information overload, readers are increasingly challenged by the volume of book choices. With thousands of new titles released annually across formats and genres, finding content that aligns with individual preferences is no longer as straightforward as it used to be. In response, advanced recommendation technologies are being developed, designed to help readers discover meaningful, tailored content with greater ease. However, many rely on bestseller lists or basic algorithms, which often miss the mark in capturing personal taste.

Among these technologies, machine learning stands out for its ability to learn user preferences and evolve over time. By analyzing browsing behaviour, genre preferences, and book ratings, machine learning models can offer highly personalized and tailored suggestions that are not simply popular books with a similar genre. This not only improves user satisfaction but also encourages discovery of new authors and genres, as the recommendation is more focused on discovery of similar material rather than simply showing what is currently most popular.

The project, “Better Reads”, aims to provide readers with not just good, but better recommendations for reading. The core objective is to develop a web-based book recommendation application that uses the potential of machine learning or other algorithms to offer personalised suggestions based on individual reading preferences. Users engage with a responsive web interface where they can explore suggestions, make personalized reading lists, and find titles specifically for their tastes. By combining modern technologies with love for books, “Better Reads” reimagines how readers discover their next favourite story. No more endless scrolling, just better reading.

2 Problem Analysis

While the access to books, and other reading materials has increased, a new challenge also appeared: oversaturated lists. Readers are now faced with a large pool of content, making it increasingly difficult to find suitable material. The result is a pressing feeling of emptiness when one wants to find a relevant book in a sea with thousands of others.

Traditional methods of discovery, like browsing online libraries or catalogues, are not exactly sufficient with the vast and overwhelming volume of options. The process of manually filtering through this information to find relevant material can be time-consuming and lead to frustration.

Additionally, readers' preferences are often personal and dynamic, influenced by factors such as mood, purpose (leisure or academic reading), and current trends. Static, non-personalized lists or generic popularity rankings fail to capture these subtleties, leaving users with recommendations that may not truly align with their interests.

2.1 Existing tools and services

Some of the existing platforms provide book recommendation systems. One of them is Goodreads [17], which relies on collaborative filtering [3] (that includes user ratings), genres and authors with their recommendations. However as much as other people's opinions are important, it leaves the unpopular books unlikely to be recommended [1].

2.2 Project Aim

This project aims to address these issues by developing an intelligent recommendation system that leverages data such as content popularity, user ratings, and reviews to deliver tailored suggestions. By providing personalized recommendations, the system intends to reduce the load on readers, streamline the discovery process, and enhance the overall reading experience in an oversaturated internet library.

2.3 Objectives

- Analysis of user preferences and search history, in order to recommend more suitable books
- Recommendation system that changes the recommendations with the user preferences

- Filtering and sorting features for the books
- Dynamic free-text search for books

2.4 Problem Statement

The lack of sufficient services that provides the users with non-biased, personalized suggestions creates a need for a book recommendation system, that would base its algorithm individually on the users and also give a chance to the non-popular titles [6].

3 Methodology

3.1 Work Methods

For the development of the project the team used Scrum [32] methodology to ensure continuous progress and adaptability to potential changes throughout the process. It allowed the team to discover and address flaws, and flexibly add tasks to the sprints regarding their correction.

3.2 Project Organization

The project was carried out in a team, with everyone contributing as developers. Some team members also took on specific roles and had certain additional tasks to handle.

To stay organized and collaborate efficiently as a group, the team decided to use Scrum. Each role, along with its responsibilities is listed below:

- (1) Product Owner – Project Coordinator, makes sure group project meets formal requirements of the course
- (2) Scrum Master / Team Leader – plan meeting agendas, keep track of tasks and come up with tasks, ensure deadlines are met
- (3) Developers – share ideas, cooperate and collaborate with team members, actively take part in the entire software development process

3.3 Development Technologies

The front-end of the application was made with Hypertext Markup Language (HTML) [18], Cascading Style Sheets (CSS) [15] and JavaScript [20]. Those technologies provide a solid foundation and a responsive user interface for the project.

As for the web application development framework, the Django [29] framework was chosen, as it provides numerous useful build-in tools. Its extensive documentation makes it a good choice for developing secure and scalable web applications. Django furthermore is a Python based framework, which was a priority when choosing among frameworks, as it is a programming language group members would like to improve in most, as well as has a wide range of open-source machine learning libraries, which are well-known and well-documented.

4 Requirements

4.1 External Project Requirements

In the description for the project, it described that at minimum the project must have an implementation of a binary search tree, autocompleted suggestions based on user input and an implementation of merge, quick or heap sort to sort the books based on ratings, reviews or publication year [4].

The more advanced requirements were to implement a graph-based recommendation system, use breadth-first search or Dijkstra's algorithm for suggestion, and implement a heap-based ranking system to show most popular books [4].

From the requirements defined by the project provider, requirements that focus on specific sortation methods and search methods were not considered as requirements for the group's project, as the group focused on the use of Django Framework's Object-Relational Mapping, and SQL queries, thus offloading the work from the application server.

4.2 Project Requirements

The requirements were decided on based on the previously mentioned external project requirements, with the addition of requirements defined by the group members based on problem analysis, which were set up and organised in two tables as seen below based on whether they were functional or nonfunctional requirements.

Functional Requirement	MoSCoW
Database: The project must have a database, which stores static book information from a dataset, user data, and potentially analysis results for recommendation.	Must
Search Among Books: The project must have a free-text search bar, where the user can type book titles, and autocomplete suggestions are shown to the user, similar to the title the user is typing.	Must

Table 4.1: Functional Requirements (MoSCoW) - Part 1

MoSCoW [22] was used to indicate the importance of each requirement. The design and the implementation were then created and built based on these requirements.

Functional Requirement	MoSCoW
Graphical User Interface: The project must have graphical user interface, providing access to the intended functionalities of the application.	Must
Sorting: The project must have a functionality allowing books to be sorted, ordered by different data (such as titles in alphabetic order, or ratings.)	Must
Filtering: The project should provide the user with the ability to filter books based on certain attributes of the books available (such as ratings, authors.)	Should
Book Recommendation: The project should provide book recommendations to the users (for example as a sorting option), based on the books' data from the chosen dataset, or incorporating user data as well.	Should
Spelling Mistake Tolerance: Within the free-text search for book titles, spelling mistakes could be tolerated, allowing the user to find the book they intended to look for.	Could

Table 4.1: Functional Requirements (MoSCoW) - Part 2

Nonfunctional Requirement	MoSCoW
User Friendly Interface: The graphical user interface of the application should be user-friendly. User-friendliness defines as intuitive interface, the user can quickly and easily find the feature they are looking for. Test via manual testing and optionally user feedback.	Should
Reliability: The application server does not crash under heavy load and high number of requests.	Should
Performance: Each singular request to the server can be resolved in under 5 seconds.	Could
Accessibility via Different Devices: The application can be accessed with any device which supports internet connectivity and web browsers.	Could

Table 4.2: Nonfunctional Requirements (MoSCoW)

5 Design

5.1 Graphical User Interface

To design the graphical user interface for the application, the design method ‘crazy 8s’ [14] was used. Different sketches were made and discussed by the group as to which design provides the highest degree of user friendliness. The final version of the combined sketches was implemented in Figma [31] to make the front-end development of the application unambiguous.

Some notable design choices made include the permanent search bar, accessible from all the pages within the web application, allowing the user to make quick searches. The colours chosen were mainly shades of brown, which includes the colour of natural paper, providing the users with a sense of nostalgia amidst the modern world. Recommendations and book visiting history were made accessible to the user through sliders, which enhance the user experience.

Refer to Appendix A for all design pictures.

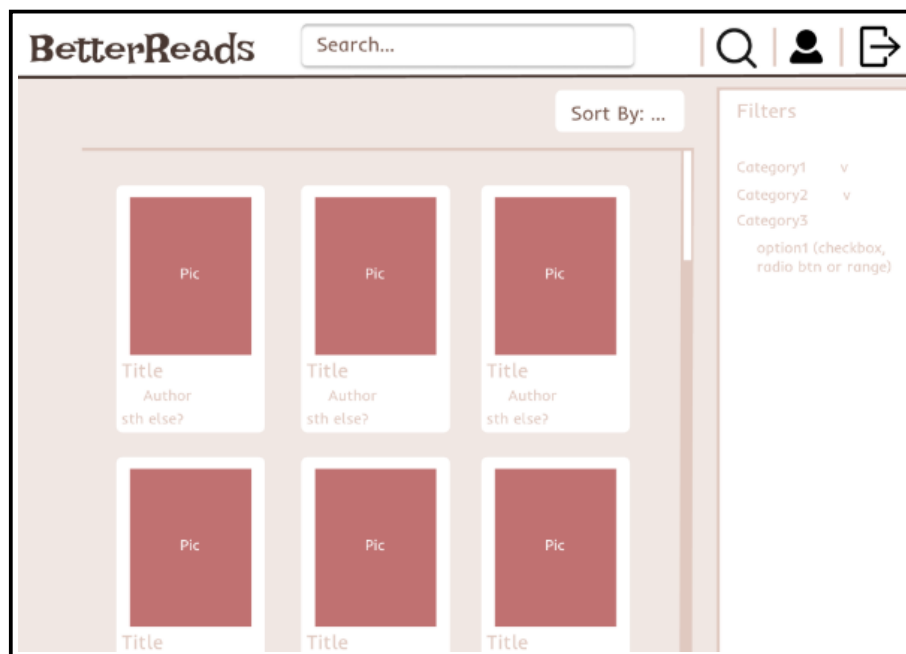


Figure 5.1: Home Page Design

5.2 Architecture

For the architecture and structure of the system, the C4 model was used to visualise it [10]. To begin with there is the system context diagram to help see the bigger picture of the system, and to clearly show the potential external systems and users.

A system context diagram is a diagram that shows the system in the center, with its users and potential external systems around it, to show a zoomed-out view that non-technical people could understand as well. [28]

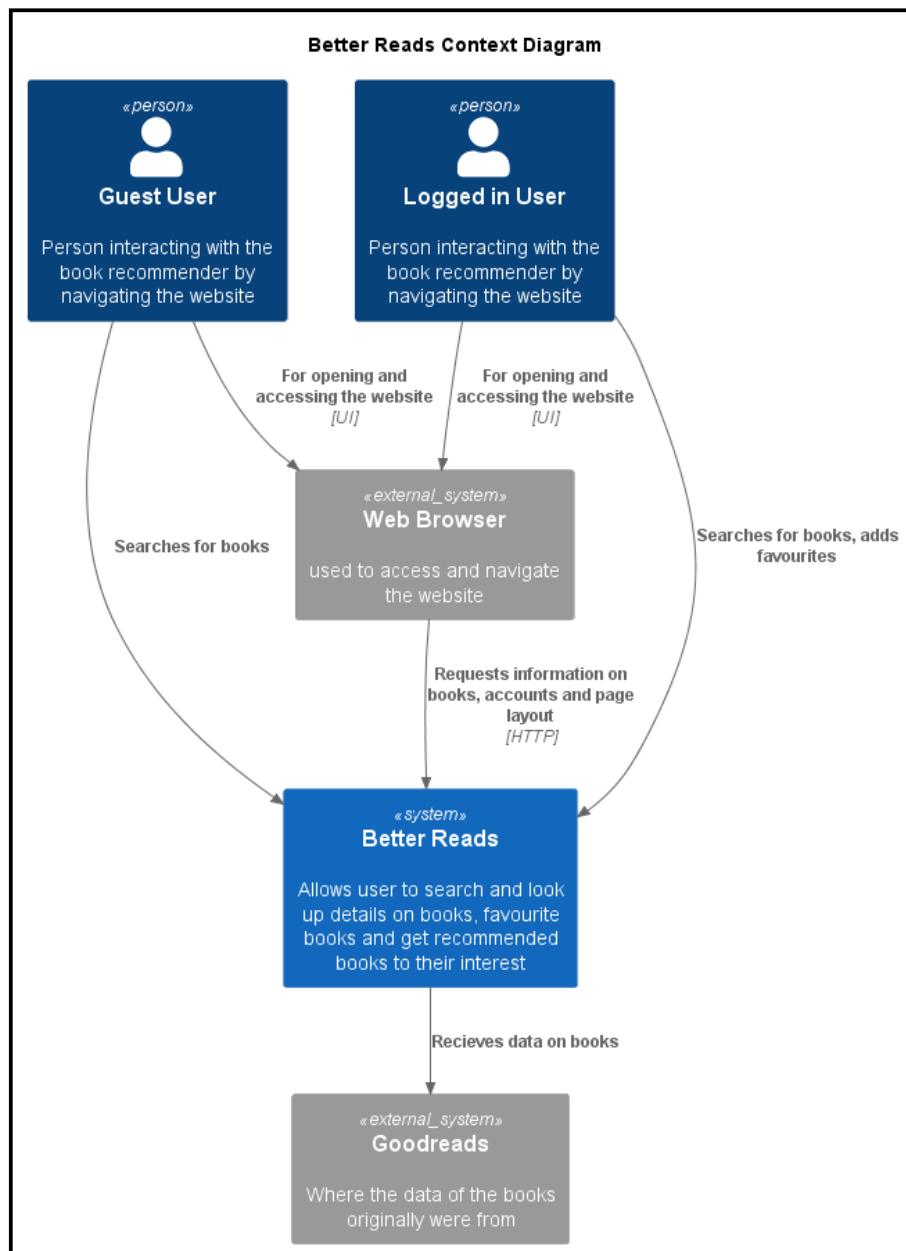


Figure 5.2: Context Diagram

First there is the users. The system has two types of users, the logged in users and the guest users. Users connect to the system through a web browser, of their choice. The web browser, using Hypertext Transfer Protocol (HTTP) [19], requests information from the system 'Better Reads', which is the project itself, and contains different containers and functionalities. Users can also search up books and add favourites to their account because of the 'Better Reads' system. The 'Better Reads' system received the data on the books from the external system 'Goodreads'.

To then get a better look at the structure of the system 'Better Reads', and in that context the structure of the system itself, a container diagram was made. A container diagram is a diagram that shows the different containers of the system, and how they communicate with each other, and it is meant to be simple, and focus on high-level technology. [13]

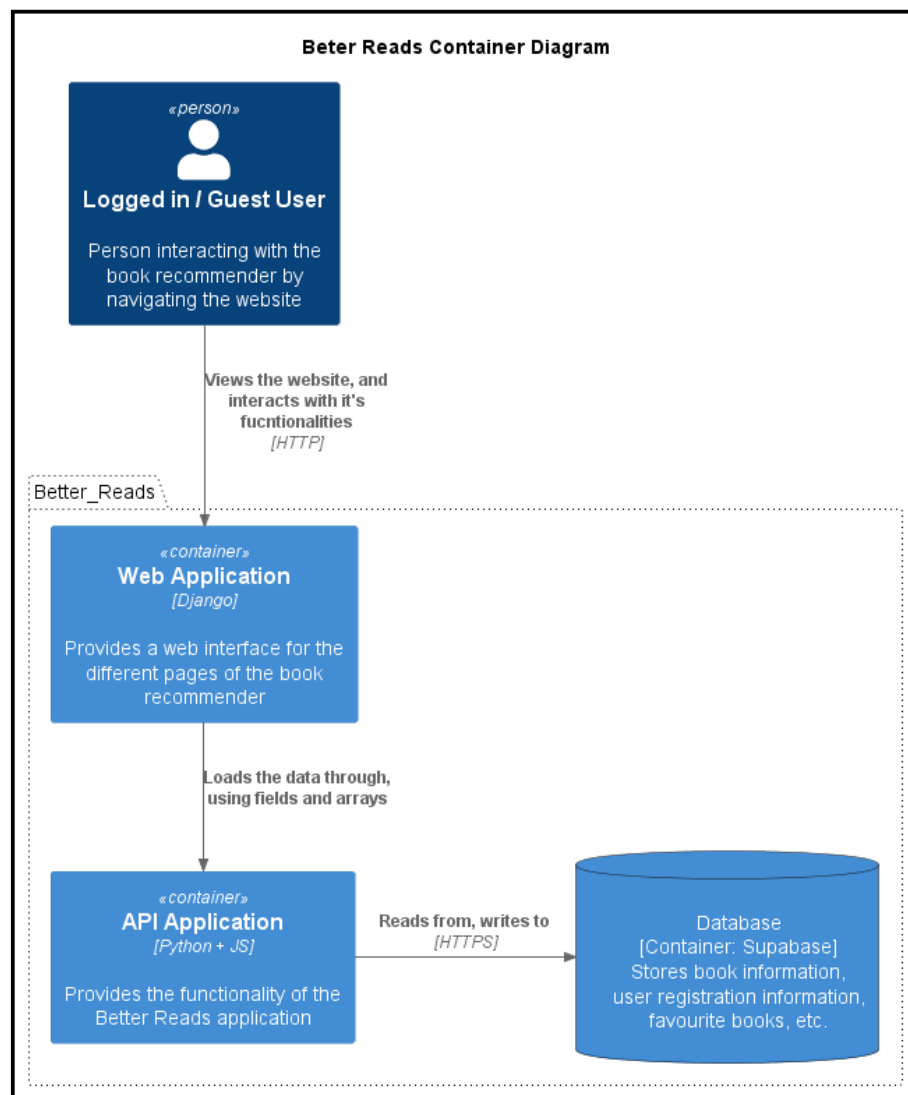


Figure 5.3: Container Diagram

Looking closer at the different containers within the ‘Better Reads’ system, there is the web application, which delivers the pages for users to view and interact with. To get the pages for the users, the web application uses the API application, which contains the different functionalities of the project. The API application then reads from and writes to the database using Hypertext Transfer Protocol Secure (HTTPS) [19]. The database is then where the details of the books and the accounts can be found.

Then there is the component diagram that allows a closer look at the structure of the API application. A component diagram is a diagram that shows the different components inside a container and each of their responsibilities. [11]

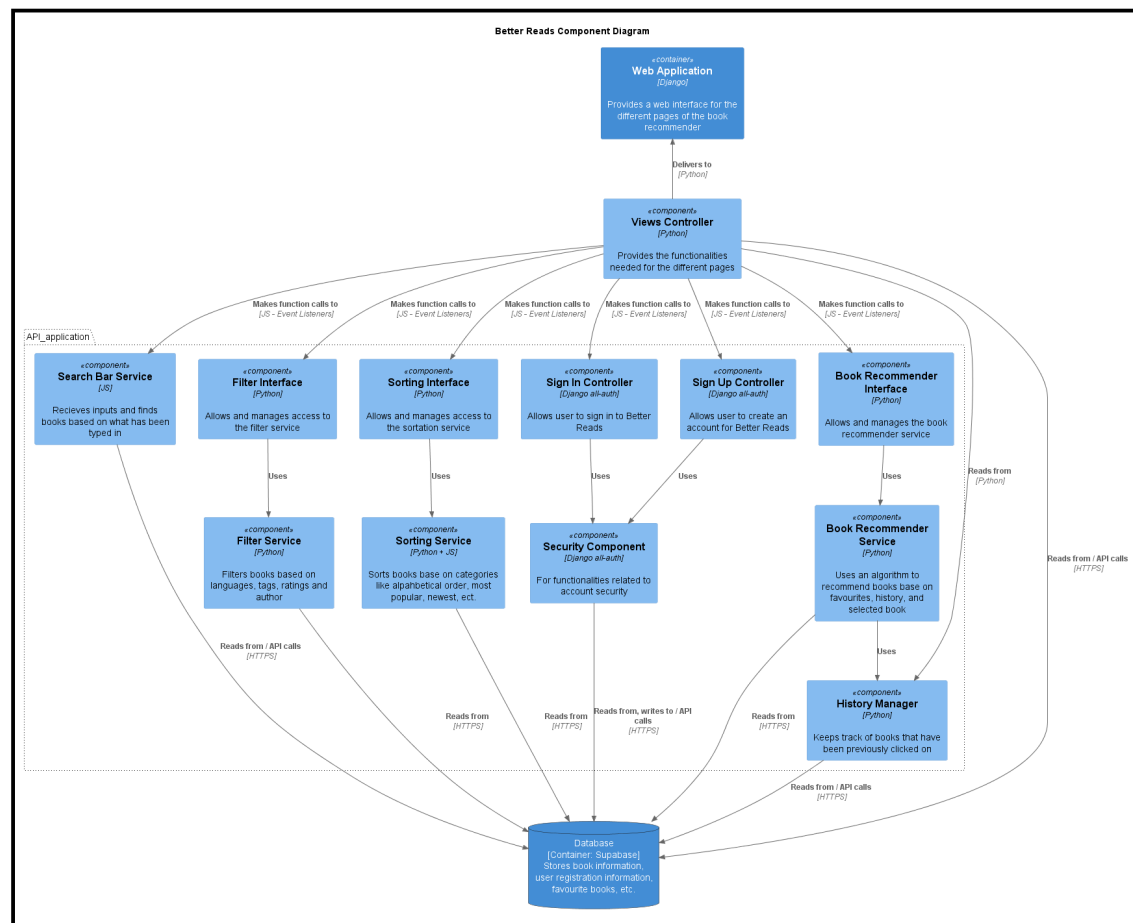


Figure 5.4: Component Diagram

The component diagram shows all the different components within the API application. To start with, there is the views controller, which takes care of handling and managing the different pages and the functionalities they need.

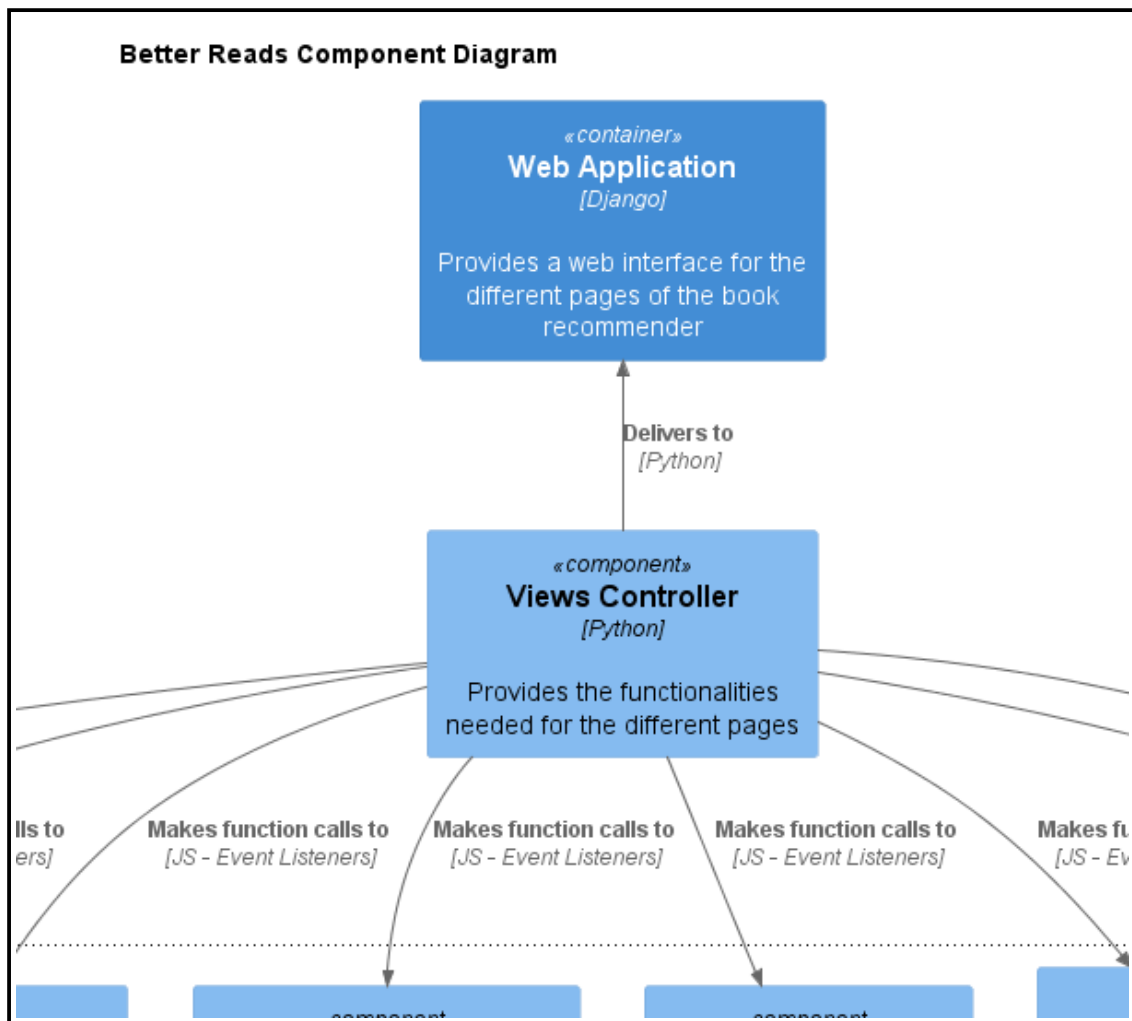


Figure 5.5: Component Diagram

The views controller makes function calls to the book recommender interface, filter interface, sorting interface, sign in controller, sign up controller and the search bar service, allowing it to use their different functionalities. The views controller also reads from the history manager to be able to display the previously visited books. To get the information on the different books, the views controller reads from database as well.

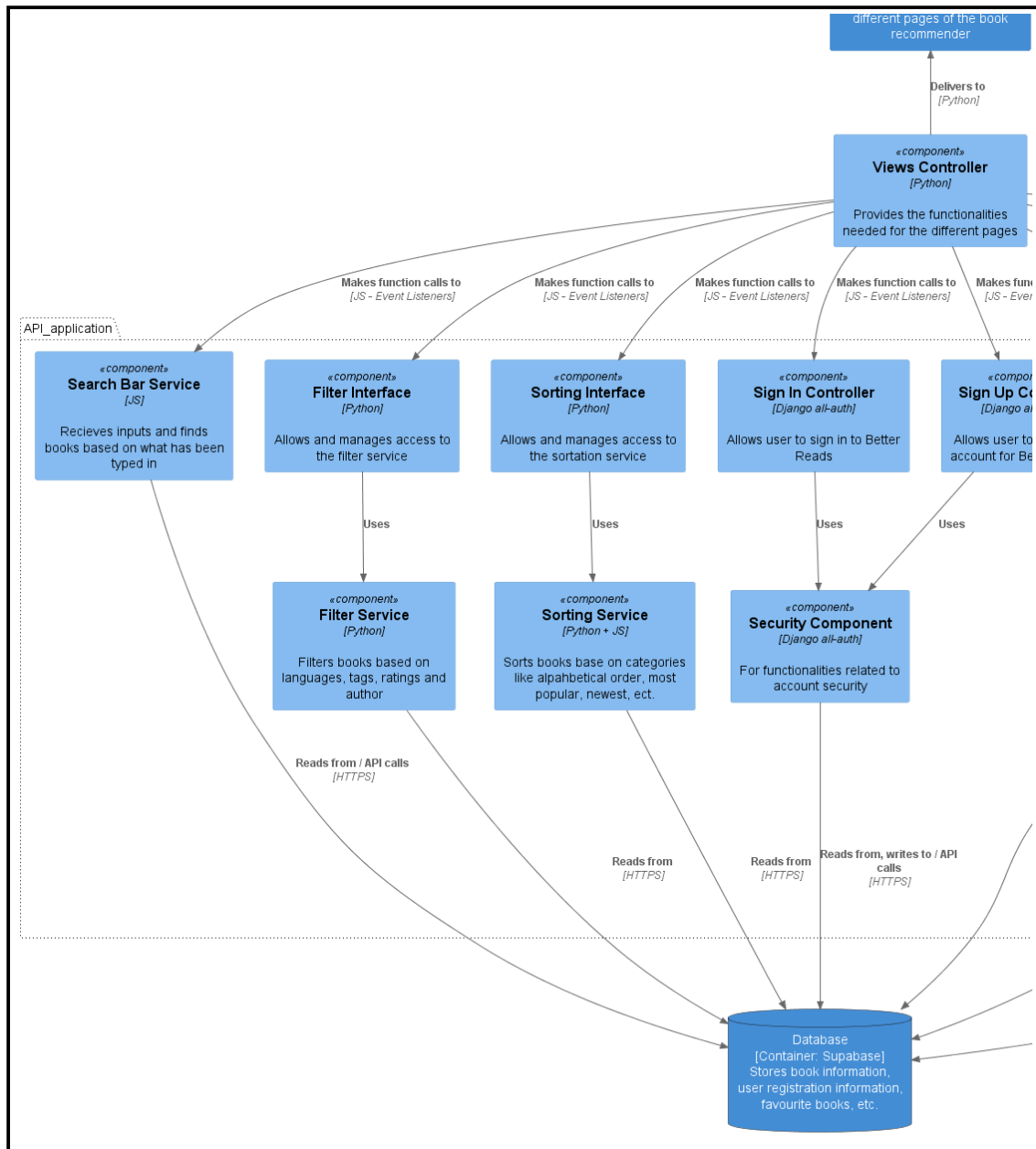


Figure 5.6: Component Diagram

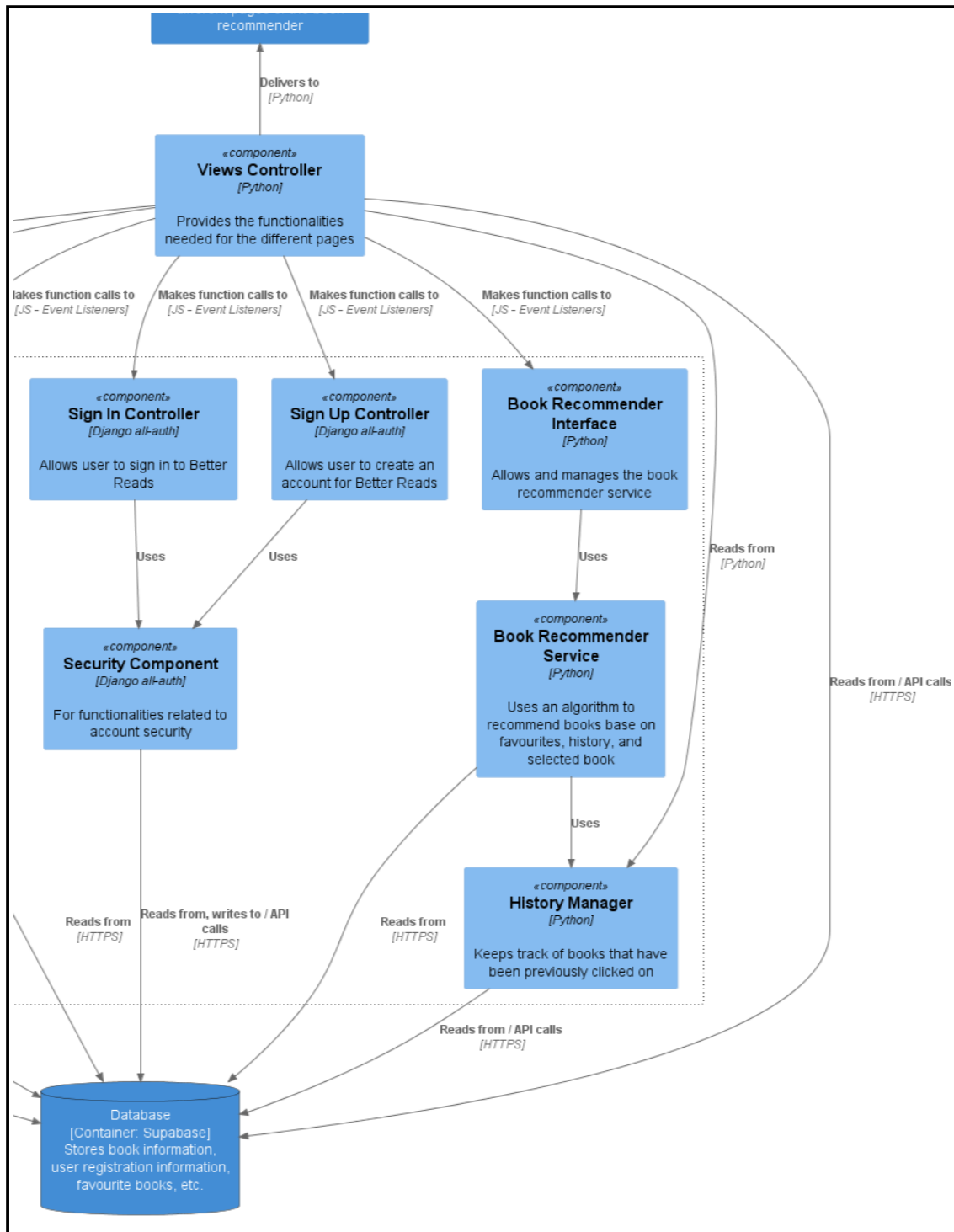


Figure 5.7: Component Diagram

The three interfaces, book recommender interface, filter interface and sorting interface, each have a corresponding service component, and they allow and manage the access of these services. For the book recommender service, it also uses the history manager, since it needs it for its algorithm of recommending books. The interfaces and their services are written using the python language.

The sign in and sign-up controllers use a security component. The security component is there for account security and for things like passwords. These three components are set up using Django and All-auth [30].

The history manager and the security component both read from and write to the database. The history manager writes to the database which books it is that previously had been clicked on, and it is also able to get the details of the books from the database. The security component writes to the database the account information and details and is able to get the account details stored in the database back.

The book recommender, filter, sorting and search bar services all read data from the database. They get the details of all the different books in the database, as well as some limited account details like favourites.

To visualise how the system could be deployed, a deployment diagram was created. A deployment diagram shows the physical nodes (e.g., servers, devices, containers) and where the different containers are hosted.

The deployment diagram [Figure 5.8] illustrates that both the web application and the API services are intended to run within a single container, which could be hosted in a cloud environment.

Users interact with the system through a web browser on a computer. The browser renders the web pages and sends HTTP or HTTPS [19] requests to the Better Reads environment. The environment contains both the web application, which manages views and user interactions, and the API services, which handles functionality such as recommendations, filtering and history tracking. [16]

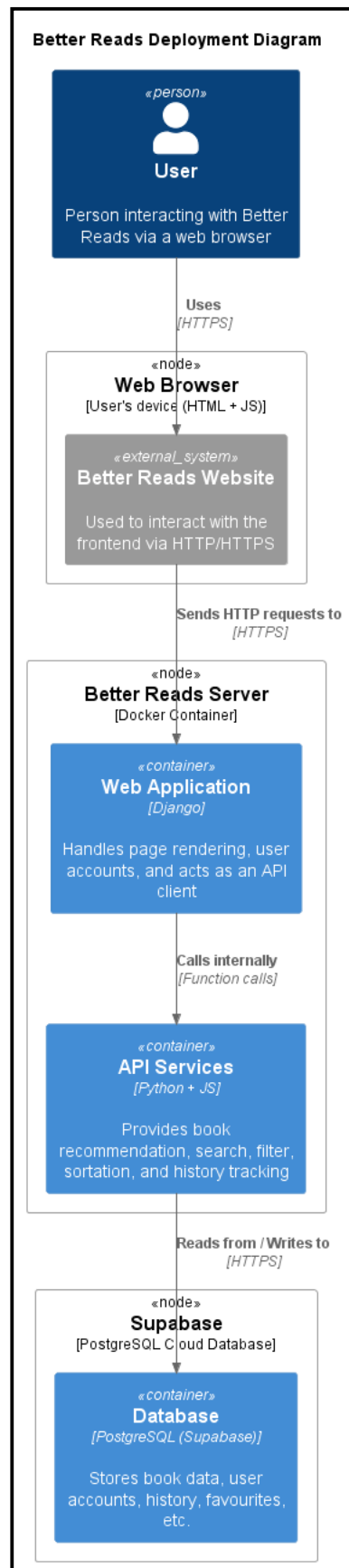


Figure 5.8: Deployment Diagram

5.3 Database and Datasheet

To store the data for the application, the cloud-based database provider ‘Supabase’ [27] is used, to take advantage of the MVC [35] architecture of Django [29]. ‘Book’, ‘BookTag’ and ‘Tag’ tables contain data from the open-source dataset [36] from Goodreads [17]. The ‘Account’ table was provided by the Django framework and its ‘AllAuth’ [30] functionality. ‘Favourites’ was used to store the books favoured by the user, and ‘UserHistory’ was used to store the last 15 book ids of book pages visited by the user, as an array.

5.4 Filtering and Sorting

Filtration can be used by the user through a side bar on the search page. Filters include:

- (1) Language: dropdown list, multiple can be selected
- (2) Authors: free text, checks for string match in database
- (3) Rating count: range
- (4) Rating average: range
- (5) Tags: free text, user picks option from suggestion list, multiple can be selected

Under the filters, a button is accessible, which can be clicked when the filters have been set to the desired settings. Then, the page reloads and displays the results for the updated query.

Sorting is accessible to the user in the search page. A dropdown allows the user to select between the sorting methods, which then automatically reloads the page and shows the results for the updated query in the requested order. The ordering algorithm used is Structured Query Language’s (SQL) Order by [34]. PostgreSQL decides which sorting algorithm to use based on the data that should be sorted, and how it should be sorted. [8]

Sorting options include:

- (1) Recommended (default, when a user is logged in)
- (2) Most Popular (default, when no user is logged in)
- (3) Least Popular
- (4) Best Rated
- (5) Worst Rated
- (6) A-Z
- (7) Z-A
- (8) Newest
- (9) Oldest

5.5 Text-based Search

The search-bar is permanently placed in the top navigation bar, that allows users to search for books based on their title from any page in the application. Suggestions are based on titles matching the user input and the first 20 closest entries are displayed in a box underneath the search-bar, that appears when the search-bar is in use. When typing out search queries a spell correction feature displays the potential correct spelling of entries to help the user.

5.6 Book Page visiting History

To create a friendly environment for the user, the last 15 visited pages of books are logged in the database and displayed at the bottom of every page, to allow them to go back to the previously visited books easily.

5.7 Book Recommendation

Book recommendations appear in three places: on the book detail page, on the account page, and on the search page with the sorting 'Recommended'. All recommendations are made based on the tags assigned to each book, and exclude books from the favourites and visit history of the user. The algorithms for each option are explained below.

5.7.1 General Algorithm

The favourite books, books in search history and (depending on the page the user is on) the book which's page the user is on are combined into a single hypothetical book (separately for each category, meaning if only favourite books and search

history exists, then 2 hypothetical books are made), adding up the tags and the count of the tags all the books have. Each category is weighted depending on the recommendation algorithm the page is using.

Based on the hypothetical books, the cosine similarity [9] of each book in the database is calculated. To calculate cosine similarity, the tags associated with each book are used.

However, popularity and average rating are meaningful attributes of books: a describing factor.

To incorporate popularity better in the algorithm, after the calculation of the cosine similarity scores from the tag counts, the logarithmically adjusted rating count score (number of ratings given for a book) – logarithmically adjusted to eliminate the advantage of the few anomalies at the higher end – is added to the cosine similarity scores. The average ratings remain unadjusted.

This way, if certain books are more similar to the goal, they can still gain a larger final score than a popular book that is not so similar. To make sure all the scores are on the same scale, both the similarities, popularity scores and average ratings are normalized.

The following formula applies to the final combination: the majority of the weight is given to similarity, as the purpose of recommendation is to suggest similar books, however, the general "tie-breaker" should be higher ratings and bigger popularity.

$$FinalScore = (0.90 * Similarity) + (0.05 * Popularity) + (0.05 * Ratings)$$

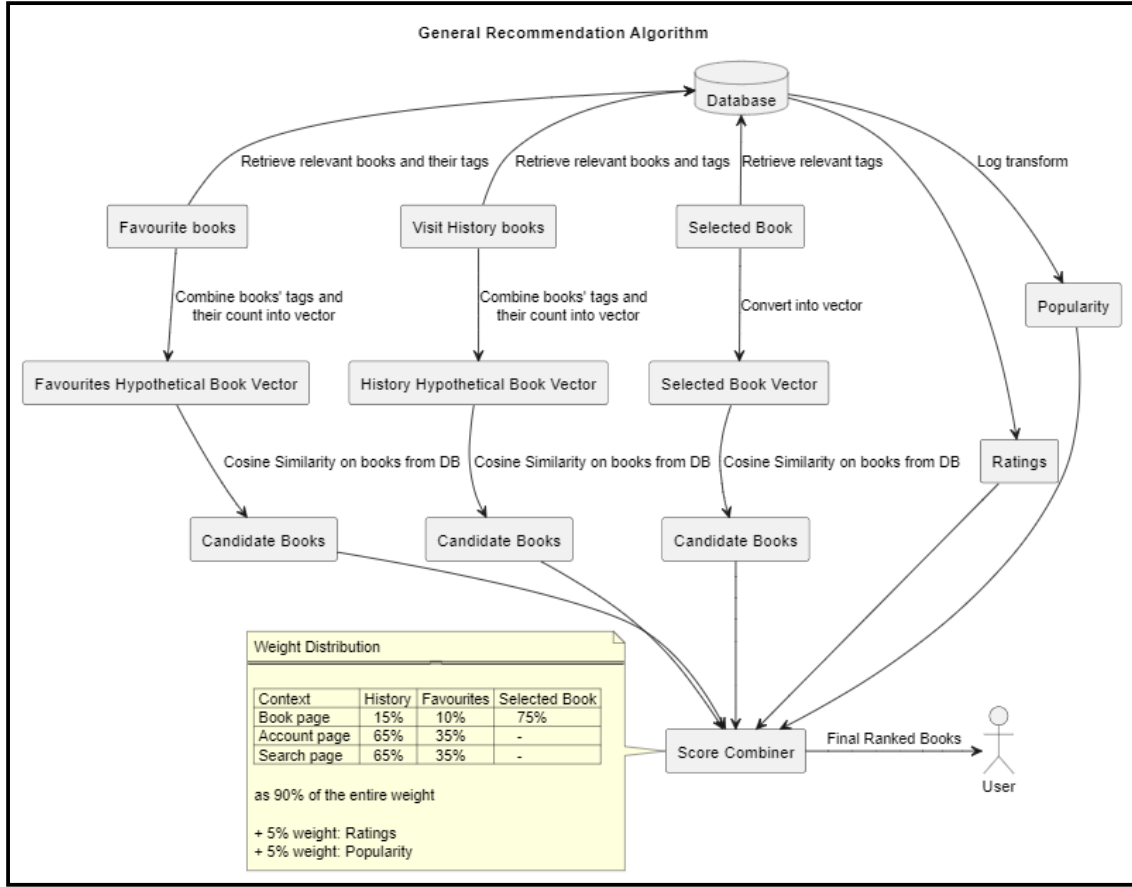


Figure 5.9: General Recommendation Algorithm

5.7.2 Book Page

First priority is to recommend books similar to the selected book. All books that share at least one tag with the selected book are retrieved, and 1000 of them are randomized. Only these 1000 books will be considered for the rest of the algorithm.

In case the user is logged in, their book visit history and favourite books will be taken into account. The value of tags assigned to all favoured books are added up, and the cosine similarity of all books compared to the sum is calculated. For the book visit history, the same algorithm applies. Then, to determine the most similar books, the similarity score in each category (selected book, visit history and favourites) is normalized between 0 and 1. For the final result, the following formula is applied to determine the score of each book:

$$FinalScore = 0.90 * [(0.75 * SelectedBookSimilarity) + (0.10 * FavoritesSimilarity) + (0.15 * VisitHistorySimilarity)] + 0.05 * [\log popularity] + 0.05 * [averageratings]$$

Moderately higher weight is given to 'visit history' similarity over 'favourites', allowing users' momentary interest to be the more defining factor.

5.7.3 Search Page

For the search page recommendation, after the filtration has been completed. To reduce the page loading and calculation time, as well as provide a variety of recommendations, 1000 books from the filtered list of books are randomized, and the scores are only calculated for these books. The user's favourites and search history are taken into account.

$$\begin{aligned} FinalScore = & 0.90 * [(0.35FavoritesSimilarity) + (0.65VisitHistorySimilarity)] \\ & + 0.05 * [\log(popularity)] + 0.05 * [Ratings] \end{aligned}$$

Higher weight is given to 'visit history' similarity over 'favourites', allowing users' momentary interest to be the more defining factor.

5.7.4 Account Page

For the account page recommendations, 1000 books from the database are randomized, excluding the user's favourites and recently visited books, and only these are used in the rest of the algorithm. Then the user's favourites and search history is taken into account. Cosine similarity regarding both categories are applied, then the final score is calculated in a way that:

$$\begin{aligned} FinalScore = & 0.90 * [(0.35FavoritesSimilarity) + (0.65VisitHistorySimilarity)] \\ & + 0.05 * [\log(popularity)] + 0.05 * [Ratings] \end{aligned}$$

Higher weight is given to 'visit history' similarity over 'favourites', allowing users' momentary interest to be the more defining factor.

5.7.5 Afterthoughts on Precomputation

In a more professional and real-world environment, the recommendation algorithms would have to be a great deal more efficient. The currently developed algorithms might compute the results in a relatively time efficient manner, however, the database only consists of 10 000 books. To become a widely used and desirable service, there should be millions of books, for which the current algorithms would take way too long to compute.

To solve this issue, pre-computation would be necessary, and a large amount of storage space. The general idea would be to pre-compute certain results, such as based on popular tags, or for every account, whenever their list of favourite books change, the ideal recommendations based on the favourite books' tags would be computed, and the list of books would be saved in the database. The next time the user would visit a page where recommendation is necessary, the algorithm would already have an almost perfectly ordered list of books to work with.

Additionally, this same method could be applied to each book, instead of accounts. Pre-compute the most similar books for each book in the database and store the ordered list – or only the top x most similar books – in the database. This way, when recommendations are made in the Book Page, computation time would be reduced to the fraction of their current computation time, only having to apply account data to this list, in case a user is logged in. Computation time would be reduced to practically zero in case no user is logged in.

Most optimally there would be a dedicated server – or multiple – updating the stored lists. The Book tags and the number of votes for each tag related to a certain book would be constantly changing – changed by the active users, real-time – therefore this dedicated server would be necessary to maintain relevance.

The current project functions as a prototype with limited optimization, as well as highly limited database storage. Pre-computing the cosine similarity for every book would result in 10 000 tables with 10 000 rows. Assuming each table is 0.5MB, 1GB of database storage would only be enough for 2 000 books, assuming nothing else needs to be stored. Due to these reasons, a different solution (randomizing 1000 books instead of using all 10 000) was utilized and the ideal, real-world solution was not attempted.

5.7.6 Afterthoughts on Machine Learning

Ideally, this project would include a machine learning component (learning defined as improving over time: the current algorithm is static, therefore does not include learning), to enhance the accuracy of recommendation. It would save the journey of users (such as which book was visited after which book) in the database and learn from it. Unfortunately, the group did not have any user data to teach the model based on, and the group had no way of generating such huge amounts of data. Therefore, machine learning was not implemented in book recommendation, instead, cosine similarity - a common machine learning algorithm component - was applied to the constant dataset of books and their attributes, providing not the best, but the achievable best results based on the data available and obtainable.

6 Implementation

6.1 Technologies

- (1) Python [25]
- (2) Django [29]
- (3) HTML [18]
- (4) CSS [15]
- (5) JavaScript [20]
- (6) Allauth [30]
- (7) Webpack [12]
- (8) Supabase-js [21]
- (9) .env [7]
- (10) Scikit-learn [26]
- (11) Pandas [24]
- (12) Numpy [23]

6.2 Authentication

To achieve a simple, but safe authentication system the pip¹ package allauth was used. Allauth comes with some simple pages for users to register and log-in. Allauth also comes with (Django) models for users and user permissions. This makes it very simple to set up since after installing the package and adding a few lines of code in different files, people can immediately access the different pages.

In the start the pages had to be accessed by writing the correct URLs in the web browser, so a button was added to allow users to login and sign-up by pressing the sign in button. The button first leads to the sign in page where the user of course can log in. On the page a link to the sign-up page was set up by being connected to the word ‘here’ on the page, and on the sign-up page the word ‘login’ then links back to the sign in page. The sign in button was made a permanent part of the top bar next to the search bar on the different pages. This same button was then set up to check if the user is logged in or not and if the user is logged in it is instead changed to be a log out button, allowing the user to log out by simply clicking on it. With Django allauth there was already password rules in place, and it also requires a unique username for each account since that is used to log in. It was then set up so that after signing up or logging in the user is then sent to the

¹Pip is python's package manager

account page as intended.

The account page is only intended to be accessed if the user is logged in, so to achieve this in the views for the account page it was set up that login is required to access the page. This way, when the button is pressed and the user is not currently logged in with an account, it leads to the sign in page instead.

6.3 Database

The Django framework makes establishing a connection to the database and handling data very easy.

To establish a connection to the database, Django needs the database language (in our case PostgreSQL [34]), a domain name or an IP address and the user credentials. While it is generally recommended to keep these details confidential, we decided that it would be secure to place the credentials in the project as the code is not intended to be publicly available. With these information in place Django will automatically establish and maintain a connection to the database when its needed.

To be able to use any data stored in the database Django requires models of the data to be created. These models contain an object's fields and their types. Django will then use these models to automatically convert the database data to python objects and the other way around.

To make the data models in the database match Django's models Django has the command line application "migration" which can be used to fetch the database tables and turn them into models or to push the Django models onto the database.

When the models and migrations have been completed the flow of the application is the following:

- (1) Django establishes a connection to the database if none exists.
- (2) Django requests the required data from the database.
- (3) Django turns the data into python objects.
- (4) The python objects are modified by the backend code.
- (5) Django requests updates to the database.

Any of these steps may be skipped if they are not needed to perform the requested action

6.4 Package Bundler (Webpack)

Modern web applications often rely on external libraries and modular JavaScript structures. However, browsers cannot natively handle imported modules such as **@supabase/supabase-js** [21]. To resolve this, the project uses Webpack as a package bundler.

Webpack processes the `index.js` file and its external dependencies, compiling them into a single JavaScript bundle suitable for browser execution. This bundling process ensures that all code, including third-party packages, is merged and optimized into a standalone script [33].

Using Webpack simplifies deployment, reduces loading times, and guarantees that no unresolved imports are left in the final production files. Thus, Webpack serves a critical role in transforming the modular development environment into a cohesive, browser-ready application.

6.5 Search Functionality

The application features a dynamic search bar that actively suggests book titles based on user input. This is accomplished by fetching book data directly from a Supabase database and comparing the user's typed input against available titles in real-time.

The logic controlling this behavior is contained within **index.js** [12]. Using the **@supabase/supabase-js** library, the script connects securely to the database, retrieves book records, and formats them into a searchable list.

To enhance the user experience, a search bar as an intelligent component is integrated. As the user types, the system runs a custom implementation of the Levenshtein Distance algorithm [2] to detect and suggest corrections for possible spelling errors. Suggested corrections are displayed subtly beneath the search input for a better user experience.

6.5.1 Spellcheck functionality

To increase user friendliness in the search bar a spellcheck feature was added. The spellcheck feature is based on a system of rules and algorithms to find the closest matches between the user input and the applications internal dictionary.

The spellcheck feature is heavily integrated with the rest of the search bar logic and uses many of its functions such as the book API, that loads the book titles and book IDs from the Database. From the retrieved titles the application generates a dictionary, that is used for comparison in the spell check function. On the lowest level the spell check feature uses the **Levenshtein distance algorithm** to calculate the smallest distance between user input and the individual words in the dictionary, to find the closest word match.

The **Levenshtein distance** between two strings a and b , denoted as $d(a, b)$, is defined as the minimum number of single-character edits (insertions, deletions, or substitutions) required to change string a into string b [5].

Mathematically, it can be defined recursively as:

$$d(i, j) = \begin{cases} \max(i, j) & \text{if } \min(i, j) = 0, \\ \min \begin{cases} d(i-1, j) + 1, \\ d(i, j-1) + 1, \\ d(i-1, j-1) + \delta(a_i, b_j) \end{cases} & \text{otherwise,} \end{cases}$$

where:

$$\delta(a_i, b_j) = \begin{cases} 0 & \text{if } a_i = b_j, \\ 1 & \text{if } a_i \neq b_j. \end{cases}$$

The Levenshtein distance algorithm itself provides the basis for the applications spellcheck feature, but has its limits, e.g. when only a single random letter is input the algorithm would still find the closest match the same goes for a lengthy input of random characters. As such a set of rules was added to apply limitations to the system. These rules include a minimum character input of at least two characters to be processed by the system and a maximum acceptable Levenshtein distance of 30% of the total character amount to ensure that inputs of random characters wouldn't be processed. Lastly another rule for direct word matching was added, so that the user could input a single word and the resulting title would contain that word. This was added because a single word input would still be processed by the Levenshtein distance algorithm and the output could at times include titles that were missing the input word but could mathematically still count as the closest match.

6.6 Filtering and Sorting

The filtering function was written in the `Filtration.py`. It takes the data for filtering from the homepage and then applies it to the books to then return only matching ones back to the page. The filtering works using URL – when the filter is applied it is also changed in the URL.

The languages and tags filters take all the possible tags and languages from the database and make sure they don't repeat on the front end. The user is able to choose multiple of them, and also able to search for them using the small search bar that appears when the dropdown is clicked, which is quite important especially for tags since there are a lot of them. The JavaScript code, using `window.onclick` method, allows the user to close the dropdown when the user clicks somewhere outside of it.

The 2 double range sliders² were made using CSS and JavaScript. One of them for average rating (using floats, with a range 0.00-5.00) and the other for rating count. The double range sliders were made combining two sliders into one using CSS. The JavaScript part is responsible for updating the numbers on the front end and also making sure the user cannot choose an impossible range (5-3 instead of 3-5) or move the sliders in other ways. The slider for rating count takes the maximum possible rating count number from the database.

The form element was added so the inputs are being submitted and the books can be filtered. The “apply” button was added, so that the user can choose their filters without the page refreshing by itself. Hidden values for the filters inputs were added to maintain the current filters state and prevent the duplicates (so e.g. filter like “tag=fantasy” will not appear twice in the URL, but it also will not disappear after clicking “apply”).

Sorting was made accessible to the user in the same side bar as filtering, with a dropdown, including all the possible sorting options, and only allowing the 'recommended' option if the user is logged in. The sorting option, when the user clicks the apply button, is added to the URL, and submits a form. Then, the Sorting service of the application is called, where the sorting is done using simple SQL queries, and the *'order_by' feature*.

6.7 Browsing History

In case a user is logged into their account, their browsing history is saved in the database. When the user visits a book's page, the user's history is retrieved from the database, decoding an array of integers, which was saved in JSON format. To this decoded array, the new book's identifier is appended, up to 15 books, meaning that if the most recently visited book would be the 16th, the least recently visited book is deleted from the array. If the newly visited book is part of the last 15

²double range slider: slider has two thumbs, thus defining a range on the slider

visited books, then the newly visited book is deleted from the list and appended again. This way it is ensured that there are no duplicated in the history of the user. Finally, the array is encoded in JSON format, and the value is updated in the database.

The user book visit history is displayed on the bottom of every page, in case the user is logged into their account. The data is retrieved from the database based on the user ID and decoded. The books are retrieved from the database using the decoded book IDs, and the list of book objects are returned to the view. In case a user visits a book's page, meaning the history should be updated while also displayed, it was made sure that the displayed list is the list of books prior to the appendment of the selected book.

6.8 Favorites

From a user perspective the favourite feature is very simple. The user can click a heart next to the book they want to mark as a favourite and it will be added to their favourite list and if they click the heart again it will be removed from the list.

Behind the scenes a few more things happen. When the heart is clicked, JavaScript code is executed to send a request to the server to add/remove a favourite book. The server will then add/remove the book from the database.

The books a user has marked as favourites can only be seen in their account page, where the favourites are statically loaded into webpage on page request.

6.9 Recommendation

To implement the recommendation algorithm defined in the design part, NumPy, Scikit-Learn and Pandas libraries were used.

6.9.1 Book Page

First, 1000 books are randomized from the database, and only these will be considered in the algorithm. Then, all tags related to the book which's page the user is on (from here on referred to as 'base book') are fetched, from which a vector is created. These steps are implemented with the help of Django's Object Relational Mapping feature, and the vector is created with Pandas DataFrame.

With the vector obtained, a pivot table is created using the randomized 1000 books, their tags and their tag counts, still using Pandas. Afterwards, the cosine similarity function of Scikit-Learn is executed, comparing the vector of the base book to the tag matrix.

As the next step, the rating count (defining popularity of the books) and the rating itself are fetched for each of the 1000 books, and both attributes are log

transformed with the help of NumPy, to lessen the effect of anomalies. Then in case the user is not logged in, the final step is the following: the popularity and rating attributes are normalized separately and added to similarity scores in a 0.05:0.05:0.9 ratio. The top 20 books with the overall highest score are returned.

When a user is logged in, their book page visit history and favourite books' data is applied to the similarity score. The tags of the books included in the book page history are combined to create an artificial book representative of all of them, then the cosine similarity function is executed for the 1000 randomized books against the artificial vector. The same algorithm is carried out for the favourite books, creating the artificial vector for all the favoured books combined and calculating the cosine similarities. The returned books do not include any of the favoured books nor the last 15 books the user has viewed.

Finally, the similarity scores (base book, history and favourites) are added up for each book in a ratio of 0.75:0.15:0.10 and normalized across all the 1000 books. The combined similarity scores are then added up with the previously calculated popularity and rating scores in a ratio of 0.9:0.05:0.05. As the last step, the top 20 books are returned.

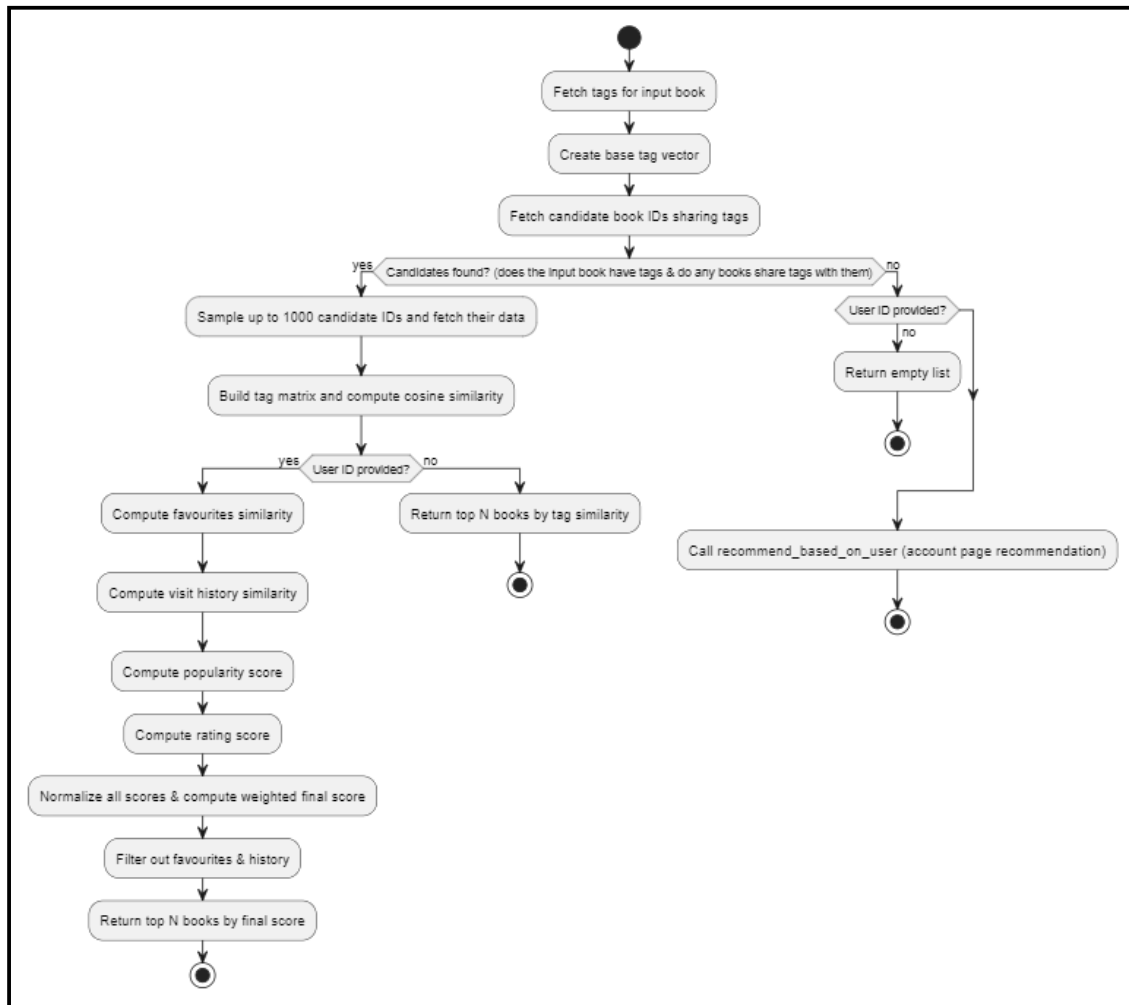


Figure 6.1: Book Page Algorithm Flowchart

6.9.2 Account and Search Page

For the Account Page and Search Page, the recommendation algorithm is only accessible for users logged in, therefore the entire algorithm is executed, in case the user has both history data and favourited books.

First, 1000 books are randomized, from the entire database in case of the Account Page, and the filtered list of books in case of the Search Page. Then, the tags are fetched for this list of books, and a tag matrix is created with the help of Pandas. As explained for the Book Page, the cosine similarity to the favourited books and books visited previously are calculated. The results are summed up in a 0.65:0.35 ratio, with more weight given to history, and normalized.

The popularity and rating scores are fetched for each of the 1000 books, logarithmically transformed separately with NumPy, and normalized separately.

Finally, the summed up and normalized similarity scores, popularity score and rating score are combined in the ratio of 0.9:0.05:0.05. The top 20 books with the highest final score are returned.

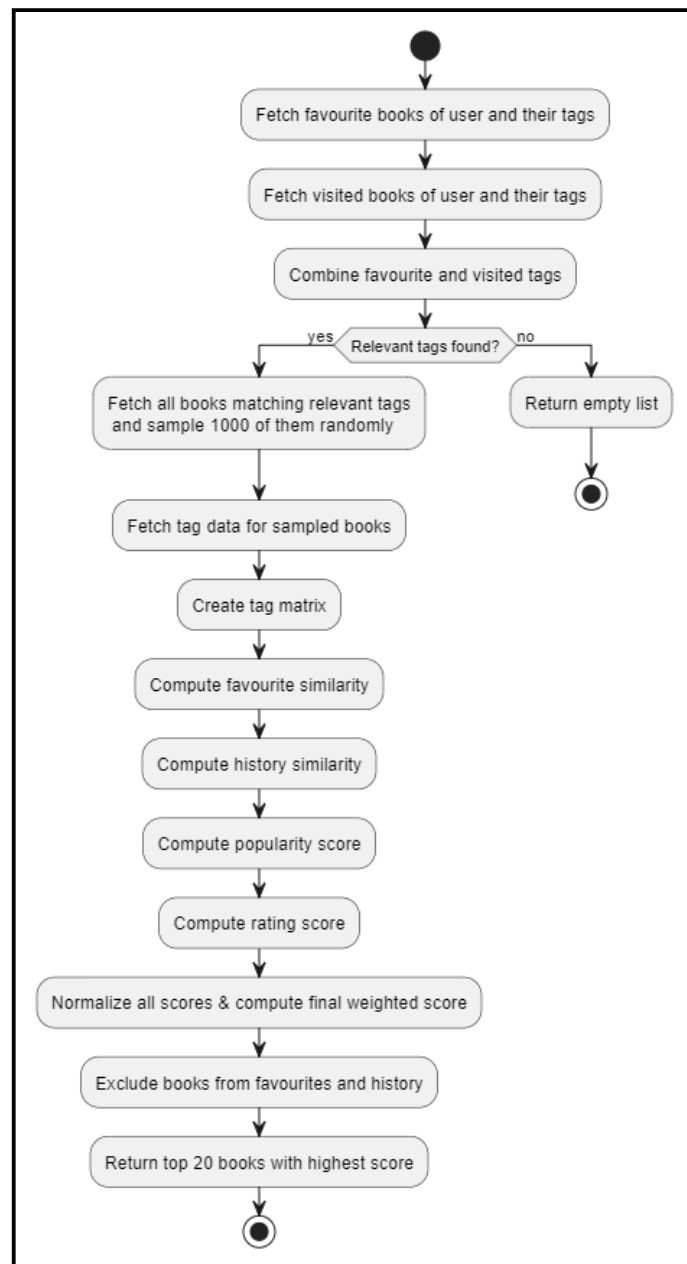


Figure 6.2: Account Page Algorithm Flowchart

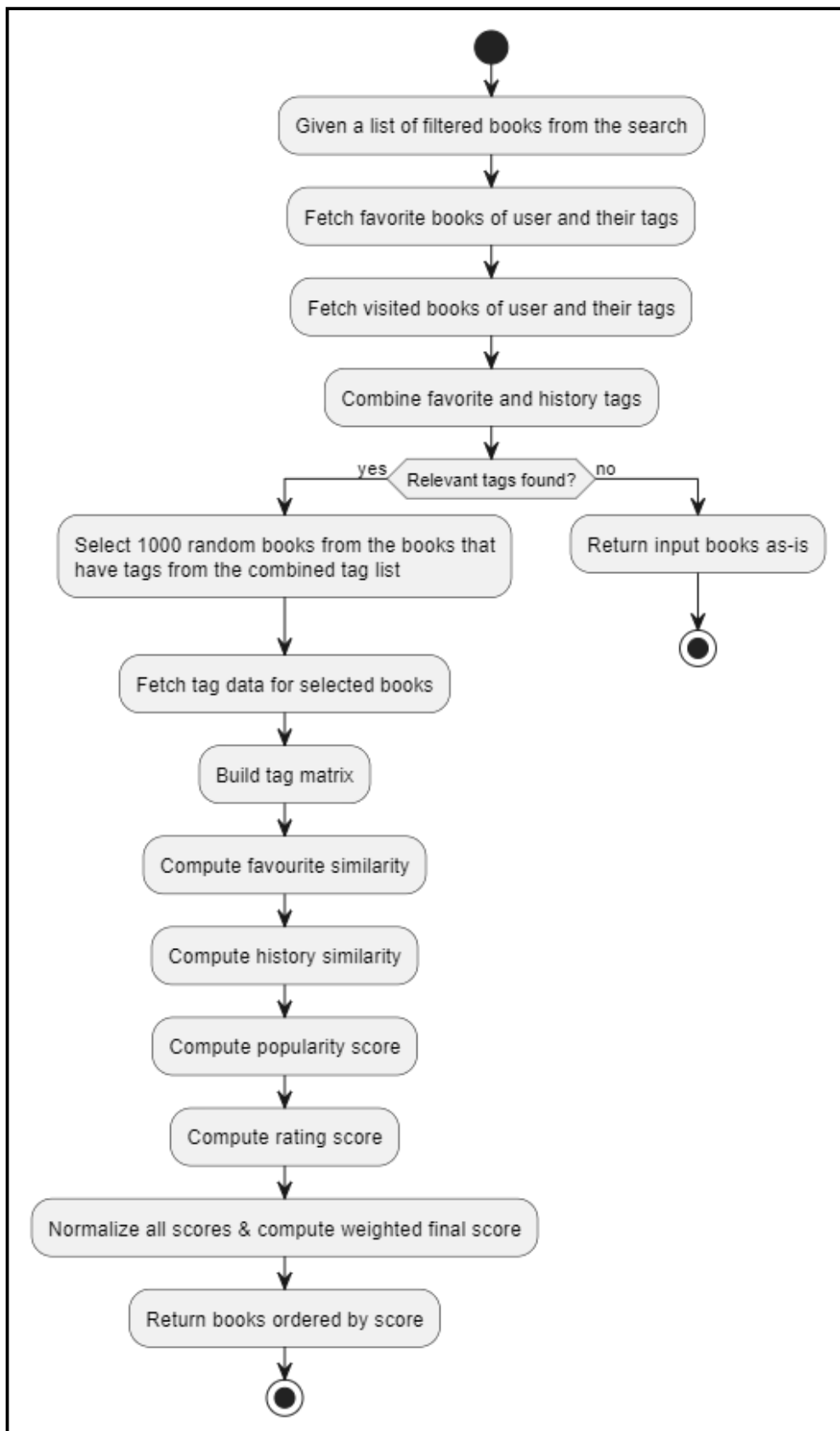


Figure 6.3: Search Page Algorithm Flowchart

7 Verification and Testing

Testing was done to verify that the application works as intended. The test scripts were written based on the requirements defined in the Requirements section. Edge cases were the main focus, as it reveals as many errors in the group's code as possible.

Regarding the requirements defined by the project provider, requirements that focus on specific sorting methods and search methods were not considered as requirements for the group's project, as the group focused on the use of Django Framework's Object-Relational Mapping, and SQL queries, thus offloading the work from the application server.

Form fields included built-in validators for inputs like email and passwords, as Django automatically performs basic validation based on model field definitions. Django forms also support built-in validators such as EmailValidator and URLValidator.

Refer to Appendix E for all test scripts.

7.1 Unit Testing

This testing method was done to test individual units of the code within all of the application's services: Sorting, Filtering, History Management and Recommendation.

Automation tests were created within Django's default testing system, and coded based on the tests scripts written by the group member. To ensure the application database was inaccessible to the unit tests, a separate settings file was created and used when executing the tests. This way, the tests were run with a database stored in a local file, and deleted upon finished execution.

Below can an example unit test script be seen:

Method	Scenario Description	Input	Expected Output	Pass / Fail
<code>recommend_based_on_book_page()</code> (with user)	User with fav = book 1, hist = book 2	Shared tags: 1-2 (2) 1-4 (1) 2-5 (1) 2-6 (1) book_id = 1, user	[4]	pass

Table 7.1: Unit Test Script for `recommend_based_on_book_page()`

In the above unit test, the method within Recommender service is being tested, in the case when a user is logged in ("(with user)"). The scenario is executed with the user having the favourite book with id 1 and has book with id 2 in their book page visit history. The books that share with tags with each other are for example book with id 1 and id 2 share 2 tags with each other, while book with id 1 and id 4 share a single tag with each other. The book page the user is on is book with id 1 (the "original book"). When the method is executed with this setup, the expected output is supposed to be only book with id 4. (Because only books that share tags with the original book are considered in the algorithm, in case books like that exist. If not, then all books would be sampled.)

7.2 Integration Testing

As the next step, integration tests were implemented, testing how components work together. This was done in Django's automation testing environment as well.

The majority of these tests cover the same cases as unit tests did, with the exception that now they operate on the actual database. Some tests were added which test the integration of components Filtration, Sortation and Recommendation. History Management was not tested in how it collaborated with other components - other than the database - as all of its methods work on the database, and none of them return anything.

Below can an example integration test be seen:

Scenario Description	Input	Expected Output	Pass / Fail
Filter by language = eng, rating count > 33, sorted by recommended	Books: Book1 = eng, 100, 4.5 Book2 = eng, 50, 3.0 Book3 = ger, 75, 2.5 Book4 = eng, 33, 2.0 Book5 = eng, 33, 1.0 Book6 = eng, 33, 2.4 Book7 = eng, 33, 4.0 User fav = Book1, hist = Book2 Tags: 1-4, 1-6, 1-7, 2-5, 3	[5, 7, 6, 4]	pass

Table 7.2: Unit Test Script for `recommend_based_on_book_page()`

This integration test validated filtration, sortation and recommendation working together. First, filtration is applied, filtering for book with English language, and rating count above 33. Then, the sortation is set to recommended, which automatically uses the recommendation service. After filtration, book 3 is excluded. Book 1 and 2 will not be recommended, as the user has recently viewed one, and has favourited the other. Thus, books 4, 5, 6 and 7 are ordered in a way that: book 5 shares a tag with history book, which is heavily weighed, and becomes the most recommended book; book 7, 6 and 4 come in this order, as they share the same amount of tags with the favourite book of the user, but this is the descending order of their average ratings.

7.3 Performance and Reliability Testing

To test performance and reliability of the application server, automation tests were written in the same way as for unit and integration testing.

Performance test cases check if each possible page request (specific Uniform Resource Locator (URL)) loads in under 5 seconds. Most of the pages loaded in less than 5 seconds, however, when a user is logged in, the Home page, the Home page when using filters, and the Book page take longer to load, 5.5-6 seconds on average. While these results prove the server does not meet the strict requirement of 5 seconds, the group believes 6 seconds is still acceptable for this prototype.

However, when the server is submitted multiple requests simultaneously, then the performance greatly deteriorates. Thus, the server does not pass the performance

requirement.

The reliability requirement was tested by submitting multiple requests to the server at the same time (36 requests), and the test passes in case all return code '200', meaning the request was successfully served.

7.4 Acceptance Testing

The final testing strategy the group incorporated is end-to-end testing, also known as acceptance testing, which was done manually. While unit and integration tests confirmed the methods and components worked well both individually and together, the manually executed acceptance tests confirmed that all the functionalities were correctly integrated with the front end of the application, sending and receiving data through requests and displaying the expected results. These tests furthermore verify that all intended features are accessible to the user, as the tests provide full coverage regarding the requirements.

Additionally, the search bar features (suggestions and spellcheck) of 'Better Reads', which was wholly written using JavaScript, was only tested manually, as Django's testing environment did not provide functionality to test it. Setting up a separate environment with a completely new framework would have been an overkill just to write a few tests for it.

An example acceptance test script is provided below:

Books in home page can be filtered • Open home page • Filter author for Harper Lee • Filter ratings for above 4.24 and below 4.26 • Filter number of ratings to above 3.190.000 and below 3.200.000 • Filter tags for pulitzer and read-in-school	Passed Comments: The result is "To Kill a Mockingbird" by Harper Lee as expected.
Then the book result is only the book To Kill a Mockingbird by Harper Lee	

Table 7.3: Manual Acceptance Test Script

Acceptance Testing was done with edge cases in mind. Moreover, these tests cover most of the test cases covered by the integration tests, however, with the addition of the front-end confirmation and accessibility. This resulted in complicated tests and user journeys, such as registration, logging in, switching between pages - which within integration testing would be a couple lines of code, such as to create

a user with a specific search history, or log in - needed for every test, making it meticulous, tedious, and slow.

It was decided that acceptance testing is executed manually, based on the written scripts, as setting up a front-end testing environment and coding all the written test scripts would have had a large overhead compared to the gains from it. These tests furthermore verify that the application works as expected in parts where other testing strategies may not notice issues, such as testing of user friendliness, intuitiveness, and accessibility. This is possible since the test executer is human, best representative of the target users.

To take verification of user-friendliness to the next level, third-parties were asked to execute certain acceptance tests (with supervision by a group member), to gain feedback on intuitiveness and ease of navigation within the application.

8 Conclusion

The goal of the project was to develop a user-friendly, web-based book recommendation application that intelligently calculates personalized suggestions based on individual reading preferences of the user.

The user can explore the large collection of books within the project's database, search for books, apply filters and ordering to get the desired results. Logged-in users can benefit from features such as favourites, visit history, and tailored recommendations, which change over time based on their user data.

8.1 Summary

The group managed to develop the web-based book recommendation system. The user is able to view and search for different books from the database. Furthermore, they can also filter and sort them based on various parameters such as language, or popularity. The search bar contains a suggestion feature, providing the user with possible results they might be looking for while typing in the name of the book. Creating an account on the website is also a provided feature for the users. While logged in, the user can add their liked books to their “favourites”, and the server keeps track of the history of the books they visit. Based on this data, the server is able to recommend them books that they might like. The users can view their search history on every page, recommendations show up on almost every page, and favourites are found on their account page.

The group focused on working on separate components or features separately, which made work distribution slightly challenging, as some parts of the project depended on others. Therefore it could not be guaranteed that each group member has evenly distributed tasks each sprint, resulting in some weeks with heavier workload than others. Fortunately none of this had any significant impact on the project plan the group had.

To sum everything up, all of the “must have” and “should have” requirements were fulfilled and the system is fully functional. This project allowed the group to gain useful knowledge regarding creating accurate recommendation algorithms which are helpful to the users. There is still room for improvement, such as improving the algorithms to become faster, or more accurate - although user data would much be needed. However, even though there are things that could have been done better, each group member's effort was greatly appreciated and was essential to the success of the project.

8.2 Future Work

In the future, the book recommendation system could contain even more accurate and faster recommendation algorithms, to satisfy the users' needs more and show them the books that match their taste the most. This could be achieved by collecting and storing large amounts of user data, and applying machine learning. This, however, was not implemented in this project as there was no user data to train the model on. Additionally, the 1GB of storage space would not be enough for such an endeavor, and would be necessary to upgrade.

What could as well be implemented in the future is a book lists feature that allows users to add the books not only to the "favourites", but also to lists they could create by themselves, and maybe even collaborate with friends in them, sharing their book lists and connecting. Perhaps even recommending users to other users based on their shared taste in books, allowing the discovery of new friends with similar overlapping interests. The feature of marking books' status, e.g. "to read", "completed", etc., would also be beneficial for the potential users of the web-application.

To improve the user experience of the application 'Better Reads,' and fulfill the requirements of implementation of Artificial Intelligence in the project, one of the group members developed the concept of a user helper. In a few words, it would be a virtual assistant capable of understanding the mood of the user, and entertaining them based on that information. This was not implemented as it is not the primary goal of the semester project, and its size greatly exceeds the achievable amount in such a time frame. Refer to Appendix D for more information.

Finally, the uncovered and optional requirements could be implemented. The system could have graphical user interface specifically for different devices, especially for mobiles, to ensure that all users can freely use it on all (or at least most) of their devices. The searching could also improve on the spelling tolerance when looking for books, so that even when the user accidentally types it in incorrectly, the system still understands what they wanted to look for in the most accurate way possible.

List of Figures

5.1	Home Page Design	7
5.2	Context Diagram	8
5.3	Container Diagram	9
5.4	Component Diagram	10
5.5	Component Diagram	11
5.6	Component Diagram	12
5.7	Component Diagram	13
5.8	Deployment Diagram	14
5.9	General Recommendation Algorithm	18
6.1	Book Page Algorithm Flowchart	27
6.2	Account Page Algorithm Flowchart	28
6.3	Search Page Algorithm Flowchart	29
D.1	AI Concept Diagram	60

Bibliography

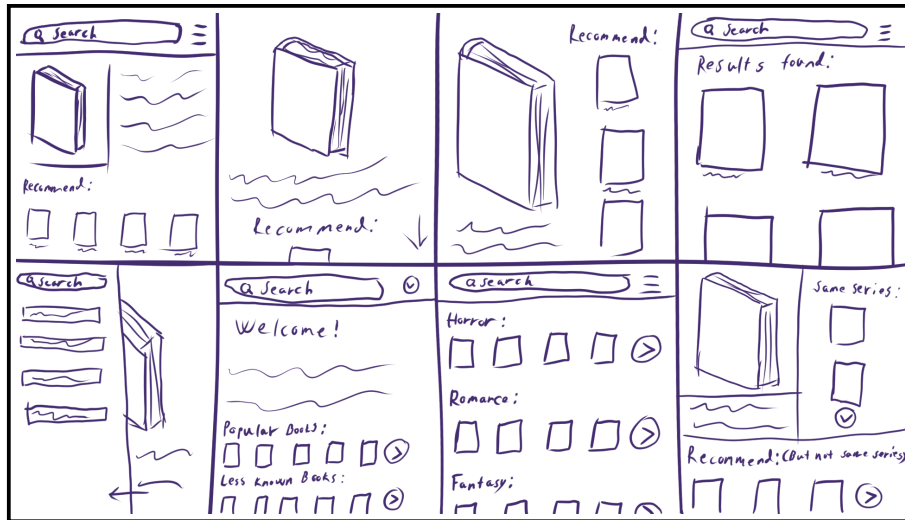
- [1] Kyusik Chung. Announcing goodreads personalized recommendations. <https://www.goodreads.com/blog/show/303-announcing-goodreads-personalized-recommendations>, 2011.
- [2] gustf. Fastest general purpose levenshtein javascript implementation. <https://stackoverflow.com/questions/18516942/fastest-general-purpose-levenshtein-javascript-implementation>, 2016.
- [3] Eda Kavlakoglu Jacob Murel Ph.D. What is collaborative filtering? <https://www.ibm.com/think/topics/collaborative-filtering#:~:text=Collaborative%20filtering%20is%20an%20information,have%20interacted%20with%20that%20item>, 2024.
- [4] Fridah Kimathi. Book recommender - efficient search and recommendation system for books - v1. <https://github.com/FridahKimathi/Book-Recommendation-System>, 2023.
- [5] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics doklady*, 10:707–710, 1966.
- [6] Mahdi Dehghan Mohammadmehdi Naghiaei, Hossein A. Rahmani. The unfairness of popularity bias in book recommendation. https://www.researchgate.net/publication/358895745_The_Unfairness_of_Popularity_Bias_in_Book_Recommendation, 2022.
- [7] Parthibakumar Murugesan. What is .env? how to set up and run a .env file in node? <https://www.codementor.io/@parthibakumarmurugesan/what-is-env-how-to-set-up-and-run-a-env-file-in-node-1pnyxw9yxj>, 2022.
- [8] Arun Nanda. Postgresql query plans for sorting data. <https://airbyte.com/blog/postgresql-query-plans-for-sorting-data>, 2024.
- [9] Katarina Supe. Understanding cosine similarity in python with scikit-learn. <https://memgraph.com/blog/cosine-similarity-python-scikit-learn>, 2023.
- [10] unknown. The c4 model for visualising software architecture. <https://c4model.com/>.

- [11] unknown. Component diagram. <https://c4model.com/diagrams/component>.
- [12] unknown. Concepts. <https://webpack.js.org/concepts/>.
- [13] unknown. Container diagram. <https://c4model.com/diagrams/container>.
- [14] Unknown. Crazy 8's. <https://designsprintkit.withgoogle.com/methodology/phase3-sketch/crazy-8s>.
- [15] Unknown. Css: Cascading style sheets. <https://developer.mozilla.org/en-US/docs/Web/CSS>.
- [16] Unknown. Deployment diagram. <https://c4model.com/diagrams/deployment>.
- [17] unknown. Goodreads | meet your next favorite book. <https://www.goodreads.com>.
- [18] Unknown. Html: Hypertext markup language. <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [19] Unknown. Http. <https://developer.mozilla.org/en-US/docs/Web/HTTP>.
- [20] Unknown. Javascript. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [21] unknown. Javascript client library. <https://supabase.com/docs/reference/javascript/introduction>.
- [22] Unknown. Moscow prioritization. <https://www.productplan.com/glossary/moscow-prioritization/#:~:text=MoSCoW%20prioritization%2C%20also%20known%20as,will%20not%20have%20right%20now>.
- [23] Unknown. Numpy documentation. <https://numpy.org/doc/stable/>.
- [24] Unknown. pandas documentation. <https://pandas.pydata.org/docs/>.
- [25] Unknown. Python. <https://www.python.org>.
- [26] Unknown. scikit-learn machine learning in python. <https://scikit-learn>

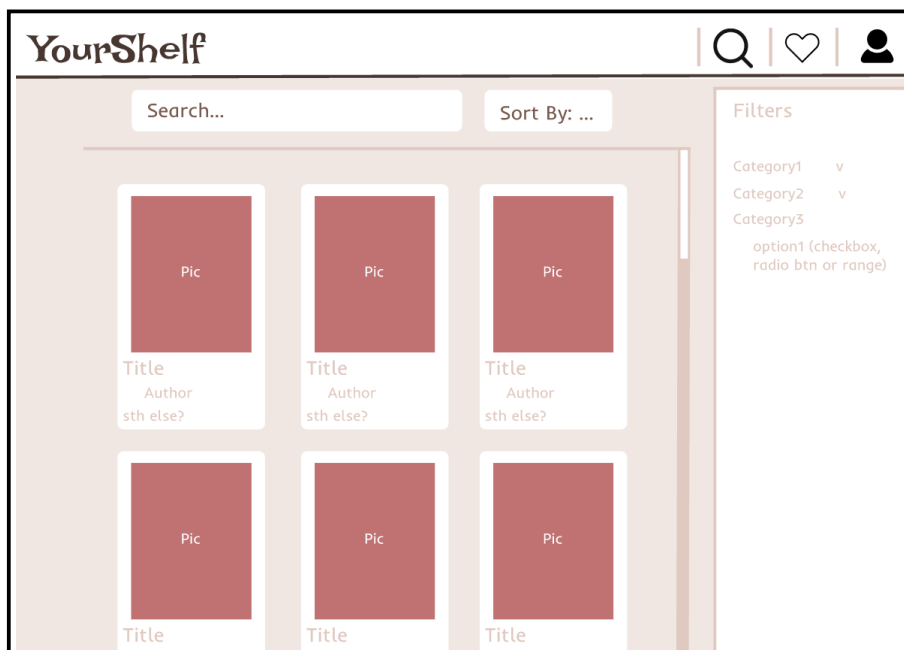
`.org/stable/`.

- [27] unknown. Supabase | the open source firebase alternative. <https://supabase.com>.
- [28] unknown. System context diagram. <https://c4model.com/diagrams/system-context>.
- [29] unknown. The web framework for perfectionists with deadlines | django. <https://www.djangoproject.com>.
- [30] unknown. Welcome to django-allauth! | allauth. <https://allauth.org>.
- [31] Unknown. What is figma? <https://help.figma.com/hc/en-us/articles/14563969806359-What-is-Figma>.
- [32] Unknown. What is scrum. <https://www.scrum.org/learning-series/what-is-scrum/>.
- [33] unknown. Integrating a modern javascript pipeline into a django application. <https://saaspegasus.com>, 2022.
- [34] unknown. Documentation postgresql 17. <https://www.postgresql.org/docs/current/queries-order.html>, 2025.
- [35] unknown. Mvc design pattern | geeksforgeeks. <https://www.geeksforgeeks.org/mvc-design-pattern>, 2025.
- [36] zygmuntz. goodbooks-10k. <https://github.com/zygmuntz/goodbooks-10k>.

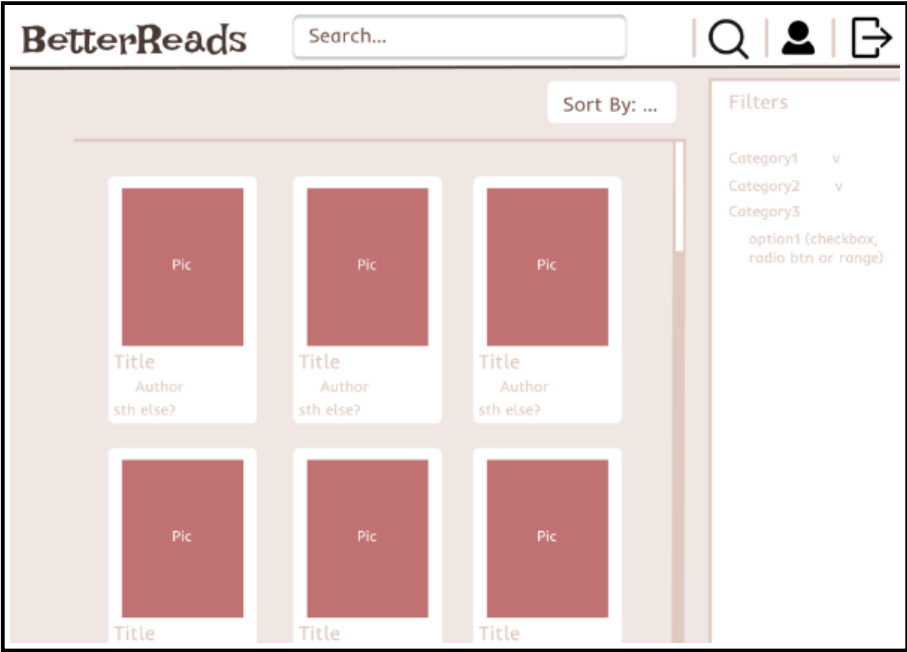
A User Interface Design



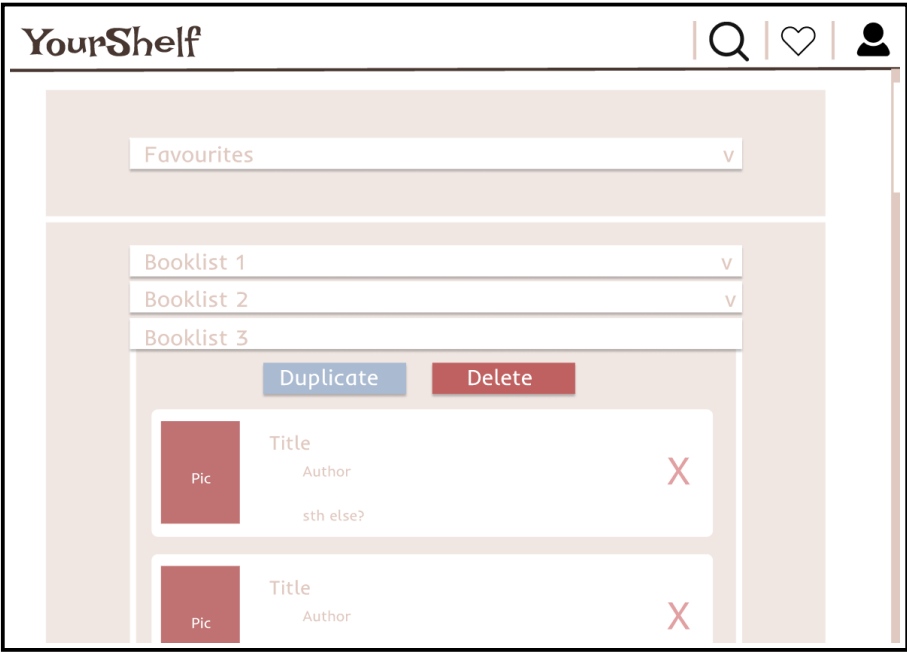
Crazy 8s



Home Page



Home Page Logged In



List Design

YourShelf

Q

♡

👤

@Account Name

Favourite Books: 34
Booklists: 5

Change Picture

Change Username

New Username:

Warning: username can only be changed every 30 days.

Change Username

Change Password

Current Password:

New Password:

Confirm Password:

Change Password

Account Page Design

B Project Proposal

UNIVERSITY OF SOUTHERN DENMARK

SEMESTERPROJECT: INTELLIGENT SOFTWARE
SYSTEMS

T630012401

Project Proposal

Group 6:

Beáta Sachs

Florian Alexander Goetz

Natalia Monika Fudali

Natasja Nicolaya Bruland

Oskar Veng Engel

Vladyslav Berezhnyi

Date:26/02/2025



Background and Motivation

With the rise of the digital age, reading remains an essential activity for both research and as a free time activity. The creation of the internet had a big impact on the way books, articles and other content could be shared, making them more accessible than ever before.

However, the mass of reading materials, shared online, presents a challenge, as the sheer volume of available content can be overwhelming for readers. Finding the right book or article to match a reader's interests and preferences can often be a long and gruelling process.

To address this issue, this project introduces an intelligent recommendation system designed to assist readers in discovering new titles. By analysing factors such as popularity, ratings, and reviews, the system can provide suggestions, making it easier for users to find content that aligns with their interests.

Aim

The group plans to develop a user-friendly software application to manage book recommendations and store the users' browsing history. This software system should be capable of handling a massive collection of book titles.

The software system will provide the user with a search function to look for specific titles, that is also tolerant towards typos and an intelligent recommendation function, that enhances the users browsing experience.

Objectives

The goal of the project is to design and implement a system for **searching and recommending** books to users by use of fundamental algorithms and data structures. This would enhance the user experience of a virtual library, allowing users to search for the specific books they have in mind, as well as getting recommended books that might be of interest to them.

- (1) **Develop an efficient search engine** that enables users to find books with high accuracy and relevance, enabling them to find desired books.
- (2) **Implement a recommendation system** based on the book that the user is currently looking at, which compares the data of books and recommends similar ones.
- (3) **Enhance user experience** by integrating an intuitive interface that supports advanced filtering (e.g. by author name, publication date, review score, etc.).
- (4) **Ensure satisfying performance** by designing a system capable of handling the currently available data without significant delays.

Initial Requirements

Requirement	MoSCoW
Database (to store book information and potential analysis results)	Must
Search among books (with autocomplete suggestions)	Must
Graphical user interface (to access functionalities)	Must
Sorting search results	Must
Filtration (to filter results by certain criteria)	Should
Book recommendation	Should
Spelling mistake tolerance	Could

Table B.1: Functional Requirements

Requirement	MoSCoW
User friendly interface	Should
Reliability	Should
Performance (results displayed within 3 seconds of search)	Should
Accessibility via different devices	Could

Table B.2: Nonfunctional Requirements

Methods

To develop the software, the group will follow scrum methodology to ensure continuous progress and adaptability to potential changes throughout the process. The team will meet regularly to ensure that everything is being implemented according to plan but there is still some time for small changes and updates based on the project's needs.

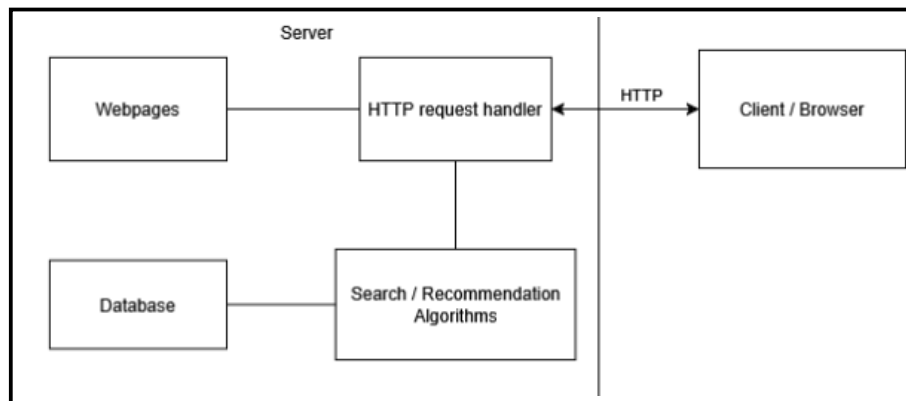
The front-end of the application will be made with HTML, CSS and Java Script to provide a solid foundation and a responsive user interface for the project. As for the back-end, the Django framework was chosen, as it provides many useful build-in tools. Its extensive documentation makes it a good choice for developing a secure and scalable web applications. Django is a Python based framework, so it encourages more maintainable code, which is incredibly beneficial for the project in a team environment.

Architecture

The software solution will be made as a web-application, meaning there will be a server that can be accessed via a regular web-browser.

The software solution will consist of the following components

- Webpages
- HTTP request handler
- Database
- Search Algorithm
- Recommendation Algorithm



Architecture Diagram

How these components connect is best described by looking at an example:

The user will first type the URL into their web-browser, this will send an HTTP request to the server requesting the default webpage. The HTTP request handler will receive the request and return the default webpage. (The default page serves as an entry point to accessing all the features our server offers).

Next the user uses the default webpage to search for a specific book, a HTTP request will be sent to the server request to search for that book. The HTTP request handler will receive the request and parse the search information to the search algorithm, the algorithm will then use the data in the database to determine which books best match the search input and return a list of books to the HTTP request handler. Lastly a webpage will be created that displays the result of the search and is then returned to the user/browser.

Risks

Throughout working on the different phases of the project's development it is expected that certain issues will be encountered. To ensure the minimization of risks it is important to identify the most common problems and take them into consideration when planning out the project, to prevent a major impact on the success of the project.

The risks can be project-related risks that the group may encounter while directly working on the project, and organizational risks that can arise when working in a group.

Project Organization

The project will be carried out in a team-based approach, where some members are assigned specific roles with a few responsibilities, while all members contribute to the project as developers.

The group decided to implement Scrum as the team collaboration framework. The expected roles, their responsibilities and assignees are the following:

- **Product Owner – Project Coordinator**
 - Ensure the team has a clear understanding of the product expectations and stays on the right track
- **Scrum Master**
 - Administer the scrum board
- **Developers**
 - Participate in all phases of software development
 - Plan sprints
 - Communicate clearly and work together with the rest of the team

Outside of the general scrum roles, the group decided to have a team leader and a secretary position. The responsibilities are the following:

- **Team Leader**
 - Ensure the team meets the deadlines
 - Plan meeting agenda
- **Secretary**
 - Administer project documentation
 - Take meeting notes

Project Plan

The project will be developed in modules and assigned to one or more group members. This approach allows for a more manageable task assignment, as the group members assigned to a module can each develop smaller parts of the module,

splitting both problems and workload.

The main modules in this case are:

The Web Application

The main application will be hosted in a local web server

- Graphical User Interface
- User Account
 - Personal History
 - Wishlist, To be read list, Reading list, Completed list, Favourites

The Search and Recommendation algorithms

Provides the application with the main logic that helps to search and recommend books, thus assisting the user during browsing.

- Sorting of search results
- Search filters
- Book recommendation system
 - Based on popularity system
- Auto correction of spelling mistakes

The Database

The data base stores the entirety of the content available in main application and should be efficient when retrieving data.

- Title Gallery

Tentative outline for the project report

Despite the project being carried out in a sprint-based manner, the group decided to only describe the final result in the report, not including the scrapped progresses. Details about sprints and meetings will be found in the appendices.

Cover Page

Shows the title of the Semester Project along with the group's name and names and student email addresses of all the team members and the group supervisor. Contains the course code, project duration period, the university name and logo, faculty, program of study and submission date.

Table of Contents

List of all the chapters with page numbers. Appendices are numbered separately.

Introduction

Overview of the project. Contains the problem statement and explains the background, primary goals and objectives of the project.

Methodology

Describes the methodologies, frameworks and models chosen by the team for the project (for example agile or waterfall).

Problem Analysis

Analyzes the project problem. Explains aims, objectives, challenges and risks.

Requirements

States the project's functional and non-functional requirements. Explains what the software should be capable of, what functionality it should have, what are the system requirements. Defines the constraints of the software. It could also contain use cases, user stories and CRC cards.

Design

Talks about the design part of the project – e.g. UI, data, component and module design. Contains UML diagrams with their descriptions and justification - what they represent, why are they relevant. It also mentions simple and incremental design, if it was used.

Implementation

Outlines the development process. Includes programming languages, frameworks, technologies and tools used. It also mentions code refactoring if it was used.

Validation and Testing

Describes how the requirements were verified and how the project was tested and reviewed.

Conclusion

Concludes the whole project. Explains what went well, what went wrong and what could have been improved. Also contains a specific contribution of each team member in the project work. Evaluation of the course and project work will also be included here.

Appendices

All the additional documents created during the project period. Such as Project Proposal, Work Agreement and Meeting Logs. Attached at the very end of the report.

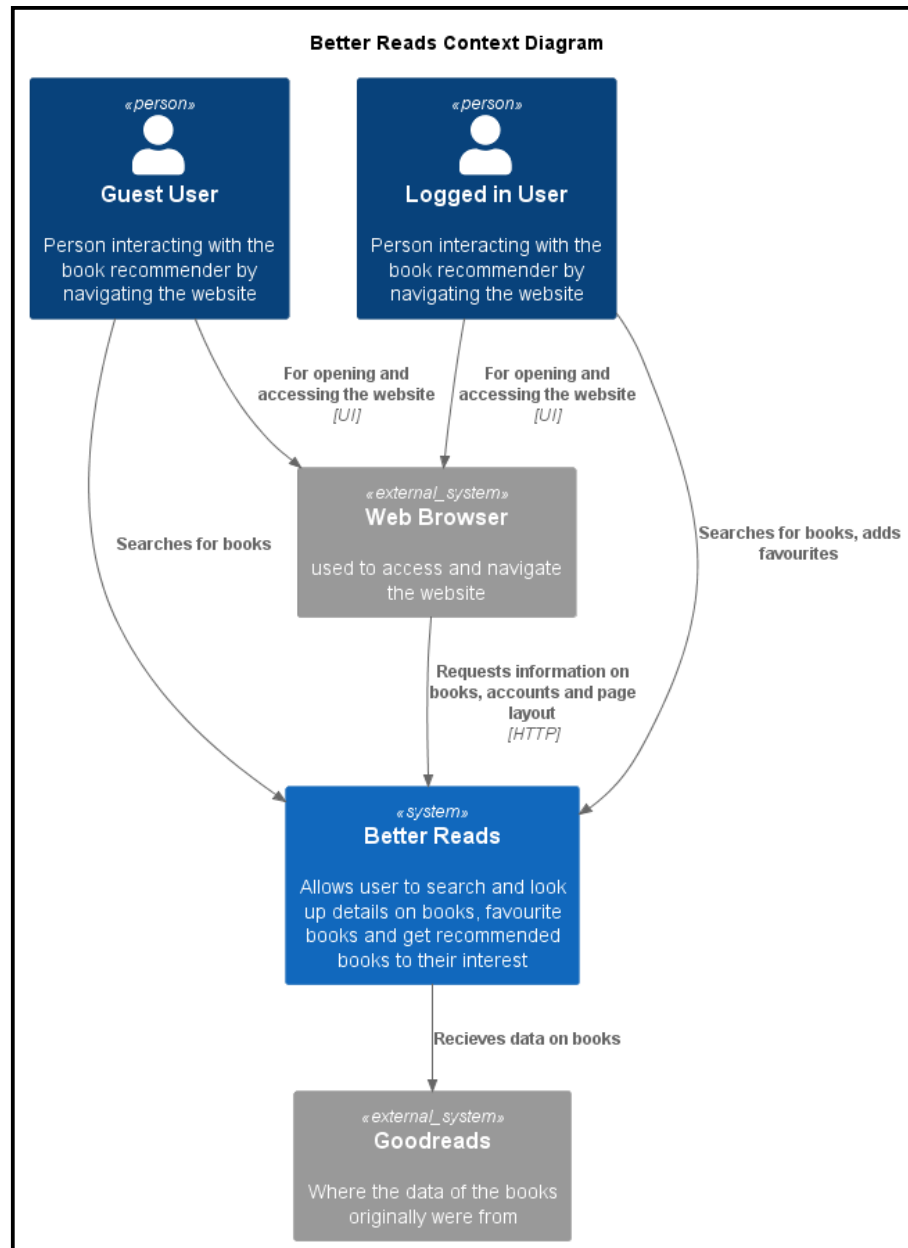
Risk	Likelihood	Impact	Response
The group misunderstanding the project's requirements	Low	High	The project description should be read thoroughly and the requirements agreed upon in the group
Missing the final deadline	Low	High	With a good time-plan and dividing the tasks evenly among the group members, this shouldn't become an issue
Delays in the time-plan	Medium	Medium	As long as it is small delays throughout the time-plan, it can be solved by doing more work the following week
Errors and bugs in the code	High	Medium	When coding, errors and bugs are common, but by testing the program thoroughly these can be found and most likely fixed in time
Miscommunication in the group	Medium	High	By having regularly scheduled meetings, as well as making sure to message and reply to messages on Discord, miscommunication should be less likely
Lack of project documentation	Medium	Medium	Organizing the project and the tasks well, and having the group members document their work throughout the project, should prevent this from becoming a problem
Unfair workload distribution	Medium	Low	Dividing the tasks evenly between the members and discussing with each other who does which tasks should hopefully make the task distribution feel fair

Table B.3: Project Risks and Mitigation Strategies

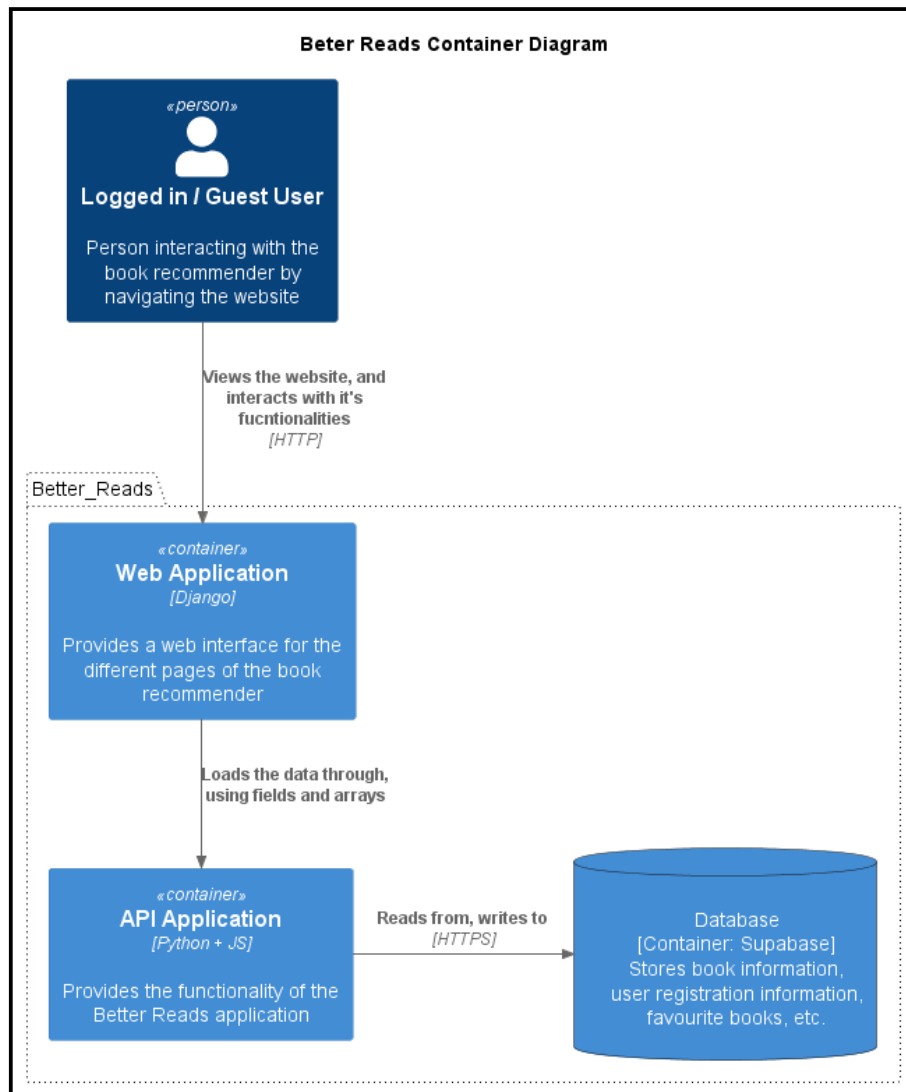
Sprint	Week	Date	Goals
1	9/10	24.02 – 09.03	Design Phase Crazy 8s Personas and Value Propositions Component Diagrams Project Proposal
2	11/12	10.03 – 23.03	Set up the server on Django Prototyping with Figma Decide on a dataset Set up the database Data modeling
3	13/14	24.03 – 06.04	Recommendation Algorithm Class Diagram Report User Profile
4	15/17	07.04 – 20.04	Search Function and Filters Saved Entry List GUI Testing Report
5	18/19	21.04 – 04.05	GUI Testing Report
6	20/21	05.05 – 18.05	Final Touch-ups on Report and Code

Table B.4: Sprint Plan and Timeline

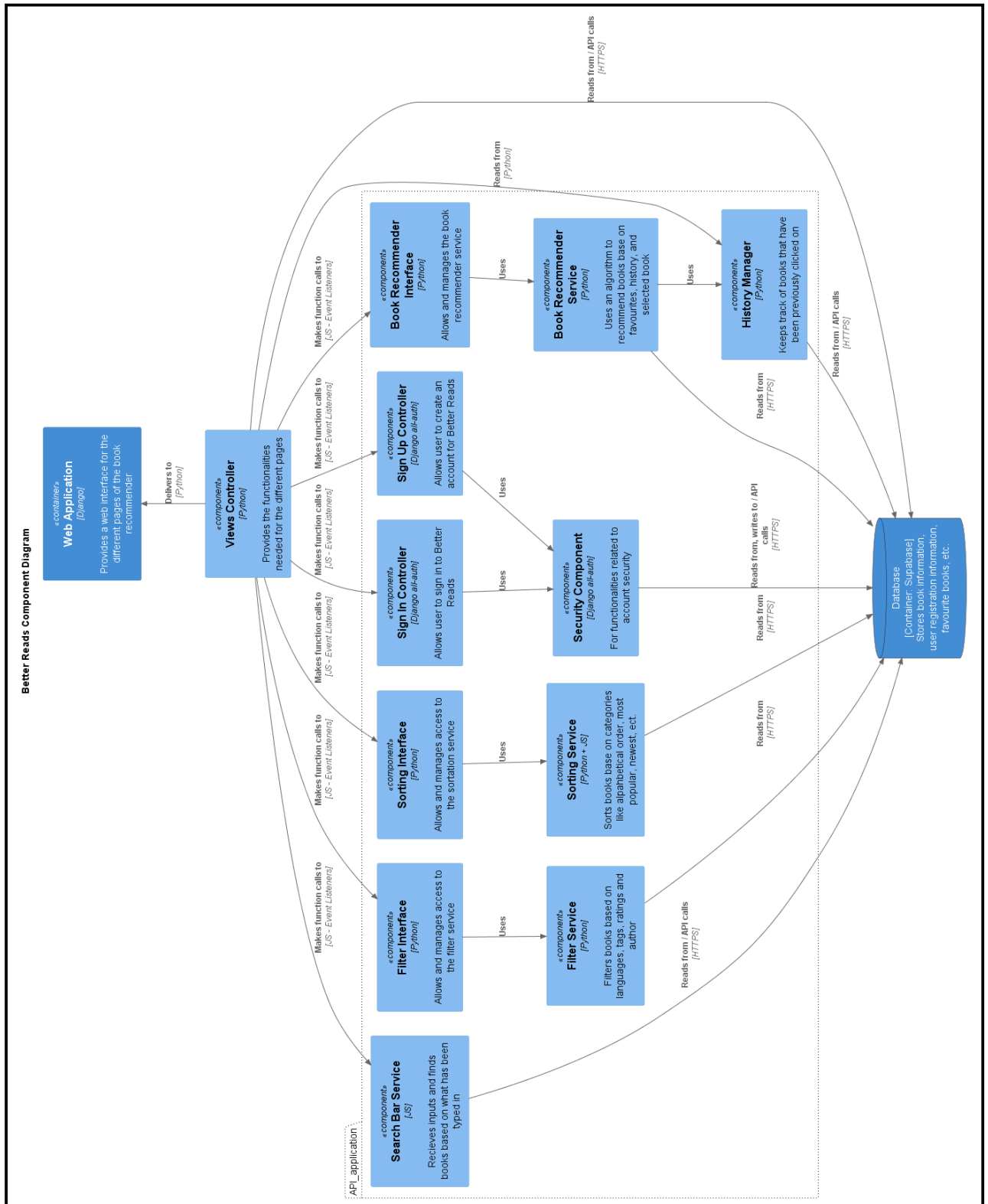
C C4 Model Diagrams



Context Diagram



Container Diagram



Component Diagram

D Conceptual Artificial Intelligence (AI) integration

As a later concept to integrate AI into the system, the question of where an AI system could be used in the application. As such the concept of an AI assistant to improve the user experience inside the application was conceptualized. The AI assistant's role inside the applications would be to assist and company the user through the application.

The AI system would track user input, such as typing speed, pauses, character deletions and used words to evaluate mood shifts in the user. Each interaction would be depicted on an n-dimensional mood graph¹ and based on the fluctuations in mood or the dominant mood, would the AI assistants interaction change. The system would use a number of personas to adapt to different moods, these personas would influence animations and response formulation to fit the mood of the user.

Initially responses would be part of a precreated template, but in time the system would start to learn new responses based on user input and forget less used or less popular responses. The response generator in turn would use the response templates and learned responses and based on the current used persona and predominant mood would output a response. These functions would be used to start the system and over time a Machine Learning (ML) component would analyze logged patterns of user interactions with the assistant to improve the systems intelligent performance. As an additional feature the idea came up give the AI assistant the ability to recommend books based on the users mood. But due to timely constraints this additional module remained in the concept stage and would be part of a future extension.

¹n = number of moods

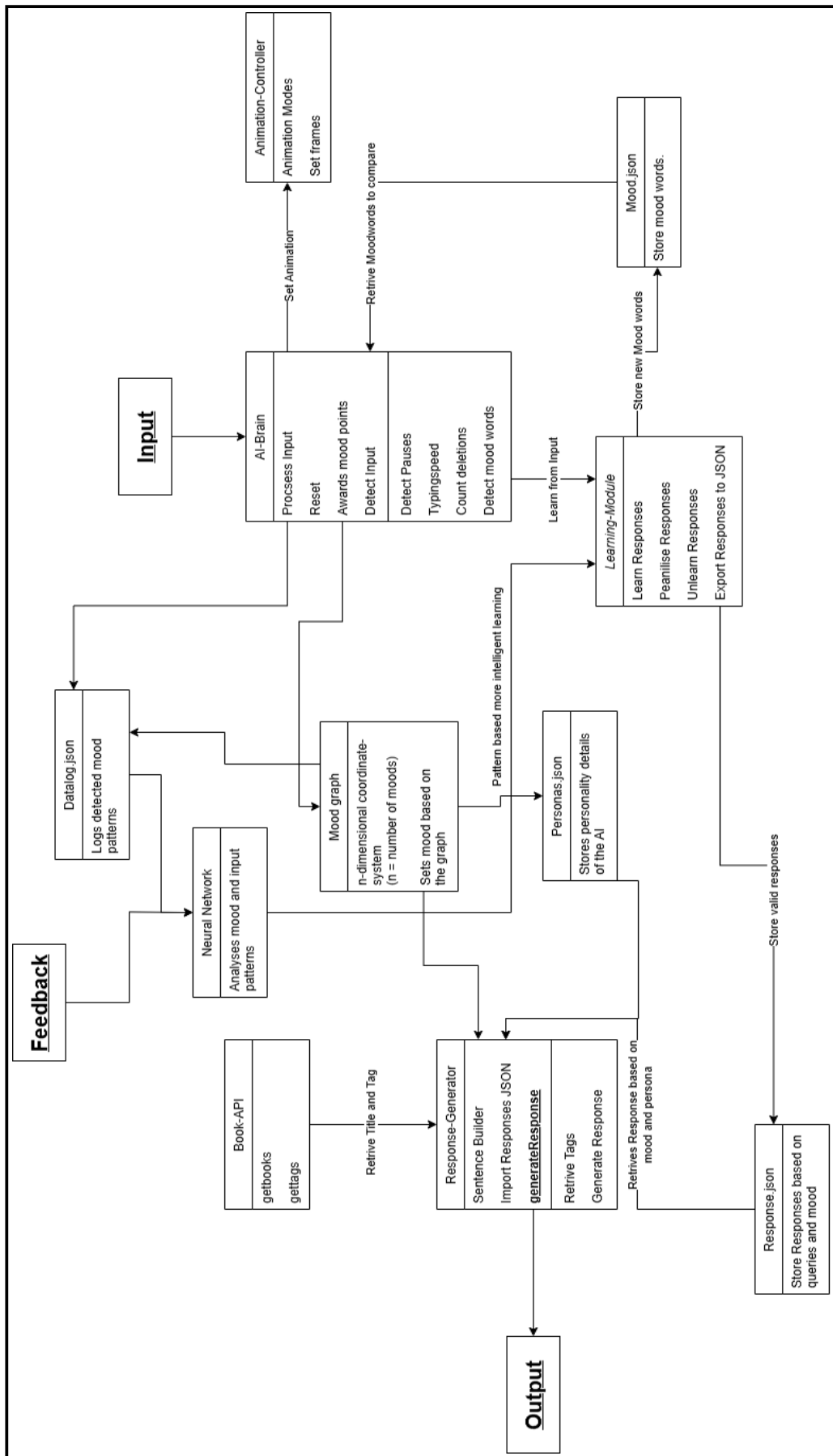


Figure D.1: AI Concept Diagram

Appendix D: Testing Script:

The testing was done with the main version (merge request #27) of the app on 6 May, 2025

Unit Tests

Sort

Order Type	Input	Expected Output Order	Pass / Fail
alpha_asc	Titles: banana, apple, carrot	apple, banana, carrot	pass
alpha_desc	Titles: banana, apple, carrot	carrot, banana, apple	pass
popularity	ratings_count: 10, 5, 20	20, 10, 5	pass
best_rated	average_rating: 10, 5, 20	20, 10, 5	pass
worst_rated	average_rating: 10, 5, 20	5, 10, 20	pass
unpopularity	ratings_count: 10, 5, 20	5, 10, 20	pass
newest	original_publication_year: 2000, 2100, 1900	2100, 2000, 1900	pass
oldest	original_publication_year: 2000, 2100, 1900	1900, 2000, 2100	pass

Filtration

Test Case	Input	Expected Output	Pass / Fail
test_filter_language	3 books with language_code='ENG', 1 with 'GER'; filter by ENG	Only 3 books with ENG	pass
test_filter_ratings_count	Books with ratings: 10, 20, 30, 40; filter between 15–32	Books with ratings_count: 20, 30	pass
test_filter_average_rating	Ratings: 1.1, 3.0, 3.3, 4.0; filter 1–3.2	Books with ratings: 1.1, 3.0	pass
test_filter_author	Authors: 'Anna Berger', 'John Anna', 'John Doe'; search 'Anna'	Authors: 'Anna Berger', 'John Anna'	pass

History

Method	Input	Expected Output	Pass / Fail
update_user_search_history()	Single insertion of book_id = 1	User history contains [1]	pass
update_user_search_history() (duplicate)	Two insertions of book_id = 1	User history contains [1] once (no duplicates)	pass
update_user_search_history() (multiple)	Insert book 1 then book 2	User history is [2, 1] (most recent first)	pass
user_search_history()	User history as [2, 1]	Returns book list ordered by [2, 1]	pass

Recommendation Helper Functions

Method	Input	Expected Output	Pass / Fail
favourites_similarity()	4 books Shared Tags: • 1-2 • 2-3 • 4	$1 > 2 > 3 = 4 = "0"$	pass
history_similarity()	4 books Shared Tags: • 1-2 • 2-3 • 4	$2 > 3 > 1 > 4 = "0"$	pass
popularity_score()	4 books Popularity = $10 * id$	$4 > 3 > 2 > 1$	pass
rating_score	4 books Rating = $5 - 0.5 * id$	$1 > 2 > 3 > 4$	pass

Recommendation

Method	Scenario Description	Input	Expected Output	Pass / Fail
recommend_based_on_book_page()	Based on tags shared with book 1 Shared tags: - 1-2 (2) - 1-4 (1) - 2-5 (1) - 2-6 (1)	book_id = 1, no user	[2, 4]	pass
recommend_based_on_book_page() (with user)	User with fav = book 1, hist = book 2 Shared tags: - 1-2 (2) - 1-4 (1) - 2-5 (1) - 2-6 (1)	book_id = 1, user	[4]	pass
recommend_based_on_user_from_list_of_books()	Fav book 1, history book 2, books 1-5 as input Shared tags: - 1-4 (1) - 2-5 (1) - 2-6 (1)	user books [1-5]	[5, 4]	pass
recommend_based_on_user()	Based on fav (book 1) and history (book 2), from all 1-6 books Shared tags: - 1-4 (1) - 2-5 (1) - 2-6 (1)	user	[5, 6, 4]	pass

Integration Tests

Sort

Scenario Description	Input	Expected Output	Pass/Fail
Sort alphabetically ascending	'alpha_asc'	First book title: '1,000 Places to See Before You Die'	pass
Sort alphabetically descending	'alpha_desc'	First book title: '美少女戦士セーラームーン新装版 1 [Bishōjo Senshi Sailor Moon Shinsōban 1]'	pass
Sort by most popular (ratings_count descending)	'popularity'	First book title: 'The Hunger Games (The Hunger Games, #1)'	pass
Sort by best rated (average_rating descending)	'best_rated'	First book title: 'The Complete Calvin and Hobbes'	pass
Sort by worst rated (average_rating ascending)	'worst_rated'	First book title: 'One Night at the Call Center'	pass
Sort by least popular (ratings_count ascending)	'unpopularity'	First book title: 'درخت زیبای من'	pass
Sort by newest publication year	'newest'	First book title: 'The Girl Before'	pass
Sort by oldest publication year	'oldest'	First book title: 'The Epic of Gilgamesh'	pass

Filtration

Scenario Description	Input	Expected Output	Pass/Fail
Filter by language code	language=FIL	Titles: "Diary ng Panget", "ABNKKBSNPLAKo?! (Mga Kwentong Chalk ni Bob Ong)"	Pass
Filter by ratings count range	minRatingCount_Selected=3974672 maxRatingCount_Selected=4780653	Titles: "The Hunger Games (The Hunger Games, #1)", "Harry Potter and the Sorcerer's Stone (Harry Potter, #1)"	Pass
Filter by average rating range	min_rating=0.00 max_rating=2.61	Title: "One Night at the Call Center"	Pass
Filter by author name	authorSearch=Chetan+Bhagat	Titles: all works by Chetan Bhagat, including "2 States", "Five Point Someone", "Half Girlfriend" etc.	Pass
Filter by tag name	tag=red-dwarf	Titles: "Infinity Welcomes Careful Drivers (Red Dwarf #1)", "Better than Life (Red Dwarf #2)"	Pass

History

Scenario Description	Input	Expected Output	Pass/Fail
Update history for new user	user_id: testuser, book_id: 1	UserHistory entry created with array [1]	Pass
Add duplicate book to history	user_id: testuser, book_id: 1 (twice)	History list still only contains [1] (no duplicates)	Pass
Add multiple books to history	user_id: testuser, book_id: 1, then 2	History list becomes [2, 1] (most recent first)	pass

Retrieve book objects from history	user_id: testuser, history contains [2, 1]	Book objects in order of [2, 1] returned	pass
------------------------------------	---	---	------

Recommendation

Scenario Description	Input	Expected Output	Pass/Fail
Recommend from book list based on user favorites	user_id: testuser (fav: Hunger Games), books with ids [1, 2, 3, 4]	Ranked book_ids: [3, 4, 2]	pass

Sort + Filter + Recommend

Scenario Description	Input	Expected Output	Pass/Fail
Filter by language and rating count, sort by alphabetical ascending	Book1 = apple, eng, 10 Book2 = banana, eng, 20 Book3 = carrot, ger, 30 Book4 = dodo, eng, 4 Rating > 32 Lang = eng Sort = alpha_asc	Apple, banana	pass
Filter by language, rating count, and sort by recommended	Lang \ ratcount \ ratavg Book1 = eng, 100, 4.5 Book2 = eng, 50, 3.0 Book3 = ger, 75, 2.5 Book4 = eng, 33, 2 Book5 = eng, 33, 1 Book6 = eng, 33, 2.4 Book7 = eng, 33, 4 User fav = book1 User hist = book2 Tags: • 1-4 • 1-6 • 1-7 • 2-5 • 3	5, 7, 6, 4	pass

Acceptance Tests

Search bar suggesting book	Incomplete Passing
1. Click on search bar 2. Enter 'day' Then the search bar suggests books that contain 'day' 3. Click on search bar 4. Enter 'الأسود يليق بك' Then the search bar suggests only one book with the name 'الأسود يليق بك'	Comments: While the search bar does suggest books by specific words, it does not suggest certain books even if typed correctly (i.e. typing "The Walking Dead, Book One" results in "The Walking Dead, Book Two", even though the book One is still recommended in the "You might like" part)

Search bar not suggesting book when not in database	Passed
1. Click on search bar 2. Enter 'abcdefgh'	Comments: No books appear
No suggestions appear under the search bar.	

Search bar suggestion redirects to book page even when incomplete	Passed
1. Click on search bar 2. Enter 'the golden li' 3. Click enter	Comments: The page redirected to the book "The Golden Lily"
I have been redirected to the book page of 'The Golden Lily'.	

Book results have hyper reference to the book page	Passed
1. Open the home page 2. Click on 'Twilight'	Comments: The page opened was the page for the book "Twilight"
I have been redirected to the book page of 'Twilight'.	

Books in home page can be sorted	Passed
Given no filters are set 1. Open the home page 2. Change sortation to Most Popular 3. Click the sort button Then the first book is ' The Hunger Games (The Hunger Games, #1)'	Comments: Sorting by "Most Popular", "Best Rated", "Alphabetical" and "Oldest Released" work as expected. "Newest Released" method had the first shown book as "Ralph S. Mouse by Beverly Cleary: Teacher Guide (Novel Units)", however the expected book "BookRags Summary: A Storm of Swords" was still

Given no filters are set 4. Open the home page 5. Change sortation to Best Rated 6. Click the sort button Then the first book is ' The Complete Calvin and Hobbes' Given no filters are set 7. Open the home page 8. Change sortation to A-Z 9. Click the sort button Then the first book is '1,000 Places to See Before You Die' Given no filters are set 10. Open the home page 11. Change sortation to Z-A 12. Click the sort button Then the first book is '美少女戦士セーラー ムーン新装版 1 [Bishōjo Senshi Sailor Moon Shinsōban 1]' Given no filters are set 13. Open the home page 14. Change sortation to Newest Released 15. Click the sort button Then the first book is ' Ralph S. Mouse by Beverly Cleary: Teacher Guide (Novel Units)' Given no filters are set 16. Open the home page 17. Change sortation to Oldest Released 18. Click the sort button Then the first book is ' The Epic of Gilgamesh'	shown and was the tenth book in the list. Due to the nature of the data we used, the only data that the sorting based on release date can use is the publication year, which the results share (Publication year: “ ”. The first change after that is the book “The Dry (Aaron Fark, #1), with the Publication Year: “2017”). The books that share their release year will be sorted differently between each other, but they will still be shown within the same group.
--	---

Books in home page can be sorted by Recommended	Passed
Given user named Test with password 12345678 exists Given user named Test has favourite book 'Princess Academy (Princess	Comments: Works as expected, the option does not show up until the user has favourited a book. Shown books based on Recommended books share tags with the Favourited books.

Academy, #1)' and no previous search history 1. Open Sign In page 2. Log in with user Test with password 12345678 3. Open Home page Then the first book result has several tags in common with Princess Academy (Princess Academy, #1) [02-fantasy, adventure, award-winner, award-winners, boarding-school, book-club, chick-lit, childhood, childhood-books, childhood-favorites, children, children-s, childrens, children-s-books, childrens-books, children-s-fiction, children-s-lit, childrens-lit, children-s-literature, classics, coming-of-age, fairy-tale, fairytale, fairy-tales, fairytales, fantasy, fantasy-sci-fi, fiction, friendship, high-fantasy, historical, historical-fiction, juvenile, juvenile-fiction, kids, kids-books, lds-authors, magic, mg, middle-grade, middle-school, my-favorites, newberry, newbery, newbery-honor, novels, princess, princess-academy, princesses, romance, royalty, school, sci-fi-fantasy, series, shannon-hale, teen, tween, ya, ya-books, ya-fantasy, ya-fiction, ya-lit, young-adult, youngadult, young-adult-fiction, young-readers, youth]	
--	--

Book recommendation for logged in user	
Given user named Test with password 12345678 exists Given user named Test has favourite book 'Princess Academy (Princess Academy, #1)' and no previous search history 1. Open Sign In page 2. Log in with user Test with password 12345678 3. Open Account page	Passed Comments: Works as expected.

Then there is a swiper with recommended books. The recommendations include books that have several tags in common with Princess Academy (Princess Academy, #1) [02-fantasy, adventure, award-winner, award-winners, boarding-school, book-club, chick-lit, childhood, childhood-books, childhood-favorites, children, children-s, childrens, children-s-books, childrens-books, children-s-fiction, children-s-lit, childrens-lit, children-s-literature, classics, coming-of-age, fairy-tale, fairytale, fairy-tales, fairytales, fantasy, fantasy-sci-fi, fiction, friendship, high-fantasy, historical, historical-fiction, juvenile, juvenile-fiction, kids, kids-books, lds-authors, magic, mg, middle-grade, middle-school, my-favorites, newberry, newbery, newbery-honor, novels, princess, princess-academy, princesses, romance, royalty, school, sci-fi-fantasy, series, shannon-hale, teen, tween, ya, ya-books, ya-fantasy, ya-fiction, ya-lit, young-adult, youngadult, young-adult-fiction, young-readers, youth]	
--	--

No book recommendation for logged in user with no favourites and no history	
Given user named Test with password 12345678 exists Given user named Test has no favourite books and no previous search history - Open Sign In page - Log in with user Test with password 12345678 - Open Account page	Incomplete passing Comments: While the user does not have any favourites in the list, the actual space for it remains, and nothing explicitly says that there are any recommendations.
Then there is a label for recommended books, and below the label it says no recommendations.	

Books in home page can be filtered	Passed
1. Open home page	

2. Filter author for Harper Lee 3. Filter ratings for above 4.24 and below 4.26 4. Filter number of ratings to above 3.190.000 and below 3.200.000 5. Filter tags for pulitzer and read-in-school	Comments: The result is "To Kill a Mockingbird" by Harper Lee as expected.
Then the book result is only the book To Kill a Mockingbird by Harper Lee	

Books can be favourited by the user & Favourited books are displayed in account page & Books can be unfavourited by the user	
Given user named Test with password 12345678 exists Given user named Test has no favourites 1. Open Sign In page 2. Log in as Test with password 12345678 3. Search book <book1> 4. Click on suggestion 5. I favourite <book1> 6. Open account page Then only <book1> is amongst the favourites 7. Search book <book1> 8. Click on suggestion 9. I unfavourite <book1> 10. Open account page Then no books are favourited	Passed Comments: Works as expected

Books in the User's favourites are not recommended	Passed
Given user named Test with password 12345678 exists Given user named Test has no favourites and no search history 1. Open Sign In page 2. Log in as Test with password 12345678	Comments: The book "Brave New World" was not showing during testing, so the book "The Catcher in the Rye", which was showing under 1984 during the majority of testing, was used, and works as expected.

3. Search book '1984' 4. Click enter Then book 'Brave New World' is amongst the recommendations. 5. Click on 'Brave New World' in recommendations 6. I favourite 'Brave New World' 7. Delete 'Test' user's entire search history from the database 8. Search book '1984' 9. Click on suggestion Then book 'Brave New World' is not amongst the recommendations.	
---	--

Books in the User's search history are not recommended	
Given user named Test with password 12345678 exists Given user named Test has no favourites and no search history 1. Open Sign In page 2. Log in as Test with password 12345678 3. Search book '1984' 4. Click on suggestion Then book 'Brave New World' is amongst the recommendations. 5. Click on 'Brave New World' in recommendations 6. Search book '1984' 7. Click on suggestion Then book 'Brave New World' is not amongst the recommendations.	Passed Comments: The same exact comments as during the previous test.

Book recommendation in Book Page based on Favourites	
Given user named Test with password 12345678 exists Given user named Test has favourite book 'Princess Academy (Princess Academy, #1)' and no previous search history 1. Open Sign In page	Passed Comments: The recommended books shown take most of their preferred tags from the book that the user is currently looking at, but their favourited books obviously impact the recommendations. Without favourites, the recommendations are similar to 1984, which is the book page used for testing.

2. Log in with user Test with password 12345678 3. Search for '1984' 4. Click Enter	With favourites, children's books are featured in recommendations as well.
The recommendations include books that have several tags in common with Princess Academy (Princess Academy, #1) [02-fantasy, adventure, award-winner, award-winners, boarding-school, book-club, chick-lit, childhood, childhood-books, childhood-favorites, children, children-s, childrens, children-s-books, childrens-books, children-s-fiction, children-s-lit, childrens-lit, children-s-literature, classics, coming-of-age, fairy-tale, fairytale, fairy-tales, fairytales, fantasy, fantasy-sci-fi, fiction, friendship, high-fantasy, historical, historical-fiction, juvenile, juvenile-fiction, kids, kids-books, lds-authors, magic, mg, middle-grade, middle-school, my-favorites, newberry, newbery, newbery-honor, novels, princess, princess-academy, princesses, romance, royalty, school, sci-fi-fantasy, series, shannon-hale, teen, tween, ya, ya-books, ya-fantasy, ya-fiction, ya-lit, young-adult, youngadult, young-adult-fiction, young-readers, youth]	

Book recommendation in Book Page based on Search History	Not passed
Given user named Test with password 12345678 exists Given user named Test has search history book 'Princess Academy (Princess Academy, #1)' and no favourite books 1. Open Sign In page 2. Log in with user Test with password 12345678 3. Search for '1984' 4. Click Enter	Comments: The recommendations from the "Animal Farm / 1984" book were obviously based on the book itself (The Great Gatsby, Lord of the Flies, Of Mice and Men, Catch-22 One Flew Over the Cuckoo's Nest, The Old Man and the Sea, The Metamorphosis) and the only reference to the only previously searched book (Princess Academy) was the "Previously visited" field. When logged out, the "You might like" field shows similar books as when logged in as a user that has searched "Princess Academy" before.
The recommendations include books that have several tags in common with Princess Academy (Princess Academy, #1) [02-fantasy, adventure, award-	

winner, award-winners, boarding-school, book-club, chick-lit, childhood, childhood-books, childhood-favorites, children, children-s, childrens, children-s-books, childrens-books, children-s-fiction, children-s-lit, childrens-lit, children-s-literature, classics, coming-of-age, fairy-tale, fairytale, fairy-tales, fairytales, fantasy, fantasy-sci-fi, fiction, friendship, high-fantasy, historical, historical-fiction, juvenile, juvenile-fiction, kids, kids-books, lds-authors, magic, mg, middle-grade, middle-school, my-favorites, newberry, newbery, newbery-honor, novels, princess, princess-academy, princesses, romance, royalty, school, sci-fi-fantasy, series, shannon-hale, teen, tween, ya, ya-books, ya-fantasy, ya-fiction, ya-lit, young-adult, youngadult, young-adult-fiction, young-readers, youth]

Search bar auto correction

1. Click on search bar
 2. Type '19i4'
- Then the suggestion '1984' is proposed to be as the correct spelling
3. I press tab
- Then the word '19i4 is changed to '1984'

Not passed

Comments: the search input is auto corrected to "1944" and when searched, redirects to a page for the book "The Diary of a Young Girl" by Anne Frank. The page itself does not feature "1944" anywhere. Searching "1984" leads to the "Animal Farm / 1984" by George Orwell as expected.

Sign Up possible

1. Open Sign In page
 2. Click on 'here'
- Then I am taken to the Sign Up page
3. Enter username 'Test', email test@test.test, use password '12345678'
 4. Click Sign Up button
- Then I get errors 'Username already taken', 'Account with email already exists', and 'Password is too common and entirely numeric.'

5. Enter username that has not been used already, email that has not been used already, and password 'PleaseWork1' 6. Click Sign Up button Then the account is created, logged in and on the Account page.	
---	--

Log In with account and Log Out Given account with username 'abc' does not exist. 1. Open Sign In page 2. Enter username 'abc' and password '1234' 3. Click Log In button Then I get an error, and aren't logged in. (Because this account does not exist.) 4. Enter username 'Test' and password '12345678' 5. Click Log In button Then I am logged in and on the Account page. 6. Click Log Out Then I am logged out and on the Home page.	
---	--

Performance and Reliability

Performance (expected load time: 5 seconds)

Page	URL	Logged In?	Actual loading time (seconds)
Homepage	'/'	No	< 5
Account Page	'/account/'	No	< 5
Book Page	'/book/2767052/'	No	< 5
Homepage	'/'	Yes	5.6 (FAIL)
Homepage + Sort	'/?sort=alpha_asc'	Yes	< 5

Homepage + Filter	'/?language=en&language=eng&language=en-US&language=en-GB&authorSearch=E&min_rating=0&max_rating=5&minRatingCount_Selected=0&maxRatingCount_Selected=2978045'	Yes	6 (FAIL)
Book Page	'/book/2767052/'	Yes	5.2 (FAIL)
Account Page	'/account/'	Yes	< 5

Reliability

Number of requests	Time of execution	All return code '200'	Evaluation
9	32s	yes	FAIL too long execution time
24	83s	yes	FAIL too long execution time
36	127s	yes	FAIL too long execution time

Appendix E: Meeting Logs

Week: 7

Present: Bea, Florian, Natasja, Oskar, Vlad

Missing: Natalia (out of town for an event)

- Agenda:
 - Project Proposal

Project Proposal (1)

- The key goal of the project proposal is to ensure that you know what you will be doing in your project
- The following elements are useful to include:
 - **Background / Motivation:** Why is this project relevant?
 - **Aim:** Short description of what you plan to achieve
 - **Objectives:** Specific technical objectives that you wish to achieve
 - **Initial requirements analysis:** Provide an initial analysis of the requirements, prioritization of the requirements
 - **Methods:** What will your approach be? e.g., Scrum. What technologies are you going to use?
 - **Architecture:** Detail different elements that are part of a software solution
 - **Risks:** What poses a risk to the success of your project? What can you do to mitigate these risks?
 - **Project Organization:** How is work distributed across the team and how do you organize your project work? Is there a project leader? Does anyone assume a special Scrum role?
 - **Project plan:** How do you divide your work into iterations to help you achieve the milestones? What are the deliverables for each milestone?
 - **Tentative outline for project report:** How will you structure your report?



♣ Background / Motivation – Florian

♣ Aim – Florian

♣ Objectives – Vlad

♣ Initial requirements analysis – Bea

♣ Methods - Natalia

♣ Architecture – Oskar

♣ Risks – Natasja

♣ Project Organization - Bea

♣ Project Plan - Florian

♣ Tentative outline for project report - Bea

- Requirements

♣ Web Application

- Framework: WordPress, Laravel, Django

♣ Noun verb analysis

♣ Search Functionalities - Must

- Implement a Binary Search Tree (BST) for fast book lookup.
- Provide autocomplete suggestions based on user input.

♣ Sorting Functionalities - Must

- Implement Merge Sort or Quick Sort or Heap Sort to sort books by ratings, reviews, or publication year.
- ♣ Recommendation Functionalities - Should
 - Implement a Graph-based recommendation system, where books are nodes and edges represent relationships (e.g., co-purchases, same genre, author collaborations).
 - Use Breadth-First Search (BFS) or Dijkstra's Algorithm for suggesting related books.
 - Implement a Heap-based ranking system to show the Top-K most popular books.
- ♣ Typo tolerance in search – Should/Could
- ♣ Database to store book information - Must
- ♣ App reqs
 - Search field
 - General (detect which column they entered and are trying to find?)
 - Advanced search (Specify that they want 4eg Author search)
 - ♣ Basically every “setting” they could have for searching, it can be customized
 - Filters
 - Sort By
 - Display
- Design some stuff already
 - ♣ Prototype
 - Crazy 8s
 - ♣ User testing
 - ♣ Diagrams
 - Sequence diagrams
 - Class diagrams
 - Data modeling
 - UML diagrams
 - Component diagrams
 - ♣ State machine
 - ♣ Use cases
 - MoSCoW
 - ♣ Personas and value propositions
 - ♣ Detailed testing :)
- Decide on things to use
- Write project proposal sections by next Wednesday (let's assign each to someone)

- Contracts: we can use the ones from previous semester

Week: 8

Present: Everyone

Missing:

- Agenda:
 - Project Proposal

Project Proposal (1)

- The key goal of the project proposal is to ensure that you know what you will be doing in your project
- The following elements are useful to include:
 - **Background / Motivation:** Why is this project relevant?
 - **Aim:** Short description of what you plan to achieve
 - **Objectives:** Specific technical objectives that you wish to achieve
 - **Initial requirements analysis:** Provide an initial analysis of the requirements, prioritization of the requirements
 - **Methods:** What will your approach be? e.g., Scrum. What technologies are you going to use?
 - **Architecture:** Detail different elements that are part of a software solution
 - **Risks:** What poses a risk to the success of your project? What can you do to mitigate these risks?
 - **Project Organization:** How is work distributed across the team and how do you organize your project work? Is there a project leader? Does anyone assume a special Scrum role?
 - **Project plan:** How do you divide your work into iterations to help you achieve the milestones? What are the deliverables for each milestone?
 - **Tentative outline for project report:** How will you structure your report?



♣ Background / Motivation – Florian

♣ Aim – Florian

♣ Objectives – Vlad

♣ Initial requirements analysis – Bea

♣ Methods - Natalia

♣ Architecture – Oskar

♣ Risks – Natasja

♣ Project Organization - Bea

♣ Project Plan - Florian

♣ Tentative outline for project report - Bea

- Design some stuff already

♣ Prototype

- Crazy 8s

♣ User testing

♣ Diagrams

- Sequence diagrams
- Class diagrams
- Data modeling

- UML diagrams
 - Component diagrams
- ♣ State machine
- ♣ Use cases
 - MoSCoW
- ♣ Personas and value propositions
- ♣ Detailed testing :)
- Decide on things to use
- Write project proposal sections by next Wednesday (let's assign each to someone)
- Contracts: we can use the ones from previous semester
- To-do until next meeting:
 - Crazy 8s
 - Personas and value propositions

Week: 9

Present: Everyone

Missing:

- Agenda:
 - During meeting
 - ♣ Check work
 - Project Proposal
 - Architecture
 - Project Plan
 - What else was there? Cannot remember
 - GUI designs – and let's decide on what to use now
 - Personas Value proposition canvas
 - ♣ Detailed Component Diagram? I'd love to have it now
 - ♣ Contracts: we can use the ones from previous semester
 - Find them now and add it to our Teams, and send it to our supervisor? Or let's set up a meeting with him asap?
 - ♣ Bea's thoughts
 - Book "playlists"? - kinda like music playlists, but books, and you could share the link to your list with your friends
 - Until next meeting
 - ♣ Set up Django
 - (I actually did it, because I won't have time in the next 2 weeks :) sowwy)

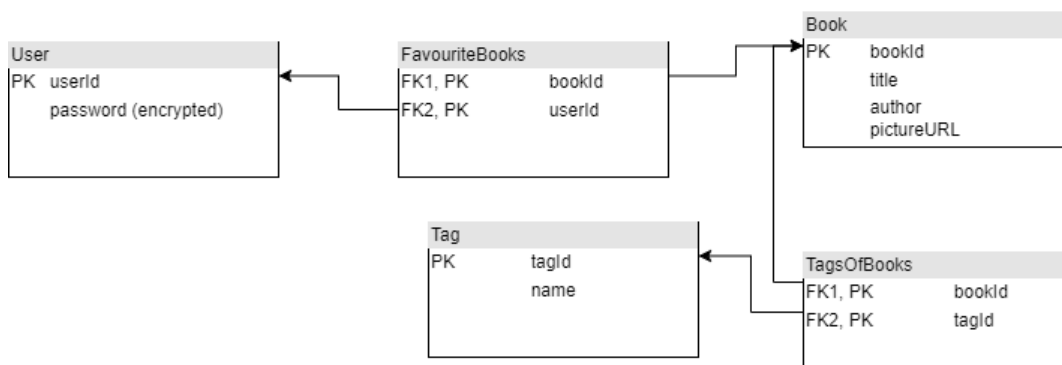
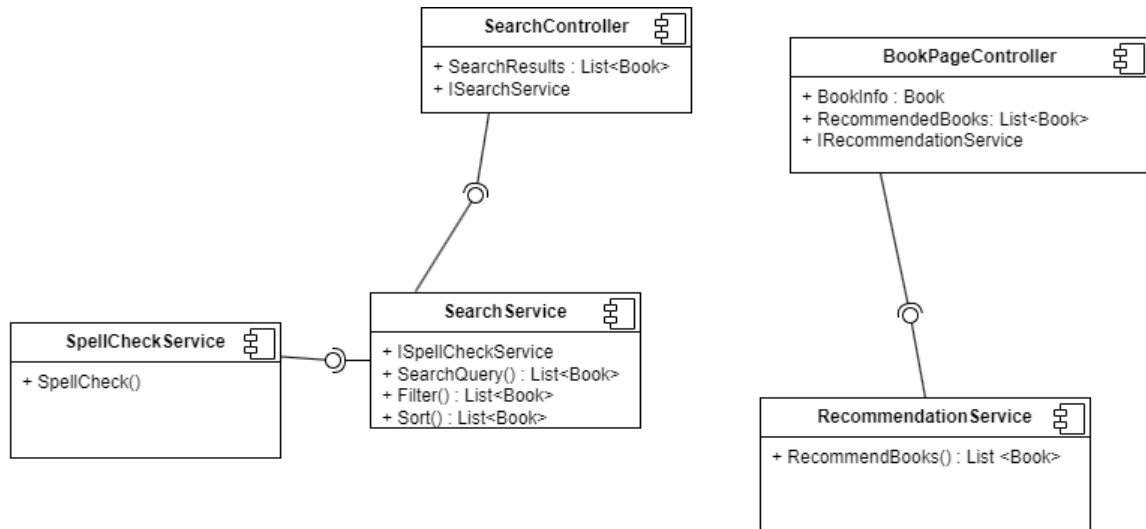
- Based on tutorial:
 - https://www.w3schools.com/django/django_getstarted.php
- Github includes virtual environment too (so the project is already 50MB) but I thought it'd be easier to have it this way than to have everyone set up their own VE and attempt to put the github reference folder in there – lmk if you'd prefer it that way
- How to set up?
 - Cmd line
 - ♣ Navigate to SPRO4 folder, then run the following command (this will start the virtual environment)
 - Scripts\activate.bat
 - ♣ Navigate to SPRO4/book_recommender, then run the following command (Django will be installed)
 - py -m pip install Django
- How to run server?
 - Cmd line
 - ♣ Navigate to SPRO4 folder, then run the following command (this will start the virtual environment)
 - Scripts\activate.bat
 - ♣ Navigate to SPRO4/book_recommender and run the following command (to run the server)
 - py manage.py runserver
- ♣ Create entire front end (every single page that we believe we need right now, use template to speed up process, and make sure it is FINISHED work by the next meeting)
 - Florian, Natalia, Vlad
- ♣ Set up database with all our available data added
 - Oskar
- ♣ Set up users (let's define what users need)
 - Natasja

Week: 10

Present: Oskar, Florian, Vlad, Natalia, Natasja

Missing: Bea

- Agenda:
 - During meeting
 - ♣ Detailed UML Diagrams!!!
 - Component diagram – how our components interact with each other, clear interfaces



Thanks you.

- Class diagram for the entire system
 - Detailed
 - Each class has clearly defined methods and properties
 - When we will get to implementation, this diagram will 100% be the only thing we rely on and allows all of us to work independently on the back end because the input and output are perfectly seamless 😊

- ♣ Peer review!
 - Deadline I think is this Friday to submit it?
 - Itslearning or outlook to find it, review it
- ♣ Contracts: we can use the ones from previous semester
 - Confirmed with supervisor
And upload it to teams
- ♣ Previous work – did everyone do it?
 - Create entire front end (every single page that we believe we need right now, use template to speed up process, and make sure it is FINISHED work by the next meeting)
 - Florian, Natalia, Vlad

Progress:

Done. Images do not load, can be fixed later because lovely Django is a special boy.

- Set up database with all our available data added
 - Oskar

Progress:

Done. Needed confirmation with the supervisor about the ability to use different datasets.

- Set up users (let's define what users need)
 - Natasja

Progress:

Found the addon for Django that seems similar to last semester's Laravel Breeze, but marginally more optimised and less visually detailed.

Supervisor Meeting notes:

- Need to do future meeting invitations.
- Focus on application architecture, diagrams etc. MAKE THE ALGORITHM OR AT LEAST START PLANNING IT!! Visual design counts much less.
- It is fine to use any dataset we want; not just the given data.
- Make the recommendation first, design the UI later if we have time.

Tasks for next week:

Peer Review:

- **Task:** Look over the G7's Proposal and make a review of it.
- **Deadline:** March 7th 23:59

Document:

- **Sequence Diagram** (page navigation for users) -- one of frontends
- **Use Cases** – Natasja (+ maybe others)

Algorithm:

- **Decide what data can be really used there:**
 - Recommend books from a description of another based on the genre (tags), rating and popularity.
 - If we collect user data, Search page (when not searching anything) will recommend based on the tags of books that you have read.

Front-End:

- **Make it more presentable.** Fix the image issues. Vlad, Natalia, Florian.

Week: 11

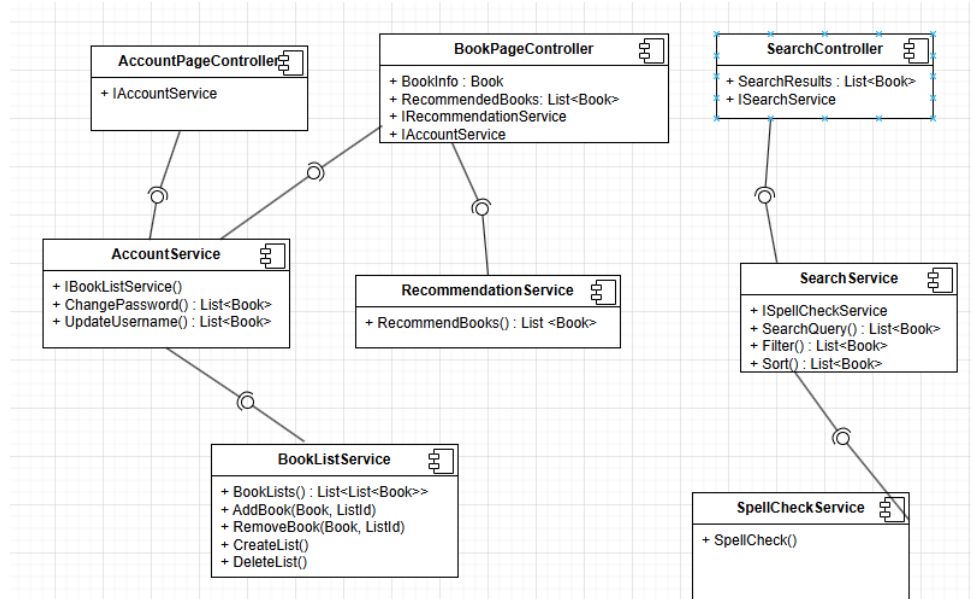
Present: Bea, Florian, Natalia, Natasja, Oskar

Missing: Vlad

- Agenda:
 - During meeting
 - ♣ Previous work – did everyone do it?
 - Document:
 - **Sequence Diagram** (page navigation for users) -- one of frontends
 - **Use Cases** – Natasja (+ maybe others)
 - Algorithm:
 - **Decide what data can be really used there:**
 - ♣ Recommend books from a description of another based on the genre (tags), rating and popularity.
 - ♣ If we collect user data, Search page (when not searching anything) will recommend based on the tags of books that you have read.
 - Front-End:

- o **Make it more presentable.** Fix the image issues.
Vlad, Natalia, Florian.

♣ Remade the component diagram



• REMARK

- o Was it viable to display images to browser via URL?
- Contracts: have you agreed with the supervisor?
- Algorithm for recommendation
 - o History of last visited 15 books in an array for user account
 - o Booklists
 - o Recommendation made based on the tags of these books, and rating count
 - o DO NOT RECOMMEND THE BOOKS IN THEIR BOOKLISTS

o Until next meeting

♣ Start implementing

- Account - Natasja
- Recommendation (Bea kinda wants this)
 - o For books under book page (detailed view)
- Search - Florian
 - o Search bar with suggestions
 - o Also front end
- Refresh without refresh (displaying books dynamically)
 - o Oskar
 - o + load more button :)
 - o By default, display all books by popularity (only 50 though)

- Filtration - Natalia
 - Language (free search, multiple choice)
 - Rating count (slider range)
 - Average rating (slider range)
 - Author (free search, accepts any string (lets say longer than 4 characters), and filters it from database via “*Steve*”)
 - Tags (free search, multiple choice)
- Sortation - Vlad
 - Popularity
 - Recommended – we can save this functionality for later development
 - Best rated (average rating)
 - Ascending (A-Z) (based on title)
 - Descending (Z-A) (based on title)
 - Newest (based on publication year)
 - Oldest (based on publication year)
 - What you need to do:
 - ♣ Create the dropdown in the front end for this, in this order as it is here
 - ♣ Create the back end (in a controller, which are placed in VIEWS, and the “views” of MVC are actually placed in TEMPLATES
 - Expect to get a list of book ids (integer), and a URL (search URL query parameters)
 - Based on those, sort the book ids based on X (implement everything except the recommended type of sortation)
- Search flow
 - Filtration -> hard search for text -> sortation

Week: 12

Present: Bea, Florian, Natalia, Natasja, Oskar

Missing: Vlad

- Agenda:
 - During meeting

- ♣ Previous work – did everyone do it?
 - Account - Natasja
 - Recommendation (Bea kinda wants this)
 - For books under book page (detailed view)
 - Search - Florian
 - Search bar with suggestions
 - Also front end
 - Refresh without refresh (displaying books dynamically)
 - Oskar
 - + load more button :)
 - By default, display all books by popularity (only 50 though)
 - Filtration - Natalia
 - Language (free search, multiple choice)
 - Rating count (slider range)
 - Average rating (slider range)
 - Author (free search, accepts any string (lets say longer than 4 characters), and filters it from database via “*Steve*”)
 - Tags (free search, multiple choice)
 - Sortation - Vlad
 - Popularity
 - Recommended – we can save this functionality for later development
 - Best rated (average rating)
 - Ascending (A-Z) (based on title)
 - Descending (Z-A) (based on title)
 - Newest (based on publication year)
 - Oldest (based on publication year)
 - What you need to do:
 - ♣ Create the dropdown in the front end for this, in this order as it is here
 - ♣ Create the back end (in a controller, which are placed in VIEWS, and the “views” of MVC are actually placed in TEMPLATES)
 - Expect to get a list of book ids (integer), and a URL (search URL query parameters)
 - Based on those, sort the book ids based on X (implement everything except the recommended type of sortation)

- o Vlad, this ain't working, please test it.
 - Contracts: have you agreed with the supervisor?
- o Until next meeting
 - ♣ Database things for user (booklists, last 15 book detail page visit of user in an array etc., what we need, let's design it now)
 - Table for favourites
 - o user_id
 - o goodreads_id
 - o timestamp
 - Oskar
 - ♣ What failed to be completed?
 - ♣ User's search history (book detail pages visited) ~15 books, displayed in a bar below on the very bottom, on every single page we have (you can use the slider I made, but don't loop it)
 - Bea
 - ♣ Account - Natasja
 - Make it pretty
 - Fix error thingy with signup
 - ♣ Connect search stuff to database, and display search results
 - Florian
 - ♣ Finish your part pls – Natalia
 - ♣ Finish your part pls – Vlad
 - If you have questions, ask Florian or Natalia, and collab with them :)

Week: 13

Present:

Missing:

- Agenda:
 - o During meeting
 - ♣ Previous work – did everyone do it?
 - Database things for user (booklists, last 15 book detail page visit of user in an array etc., what we need, let's design it now)
 - o Table for favourites
 - ♣ user_id
 - ♣ goodreads_id
 - ♣ timestamp

- o Oskar
- User's search history (book detail pages visited) ~15 books, displayed in a bar below on the very bottom, on every single page we have (you can use the slider I made, but don't loop it)
 - o Bea
- Account - Natasja
 - o Make it pretty
 - o Fix error thingy with signup
- Connect search stuff to database, and display search results
 - o Florian
- Finish your part pls – Natalia
- Finish your part pls – Vlad
 - o If you have questions, ask Florian or Natalia, and collab with them :)
- ♣ Presentation for midterm thingy – who presents what, how etc.
 - Introduction + Problem analysis
 - o Florian
 - Requirements + Design (Figma)
 - o Me
 - System architecture + Tech stack
 - o Vlad
- ♣ Meeting with Jukka – any preparation?
 - cool
- o Until next meeting
 - ♣ Favourites
 - Oskar
 - ♣ Recommendation based on user data as well
 - Bea
 - ♣ Report writing
 - Documentation – I hope everyone is doing it well 😊
 - We may want to start this after the midterm presentation or whatever it is called
 - ♣ Natalia pls ur part
 - ♣ Florian pls you too
 - ♣ Vlad + Natalia + Florian – pls merge ur stuff together so they work together *seamlessly*
 - ♣ Natasja pls logout btn

Week: 14

Present: Bea, Florian, Natalia, Natasja, Oskar, Vlad

Missing: -

- Agenda:
 - During meeting
 - ♣ Previous work – did everyone do it?
 - Why our app is soooo amazing?
 - Favourites
 - Oskar
 - Recommendation based on user data as well
 - Bea
 - Started working on the algorithm but yeah I've been sick and didn't finish, I'm sorry, I shall do this until next Wednesday
 - Natalia pls ur part
 - Florian pls you too
 - Connect search stuff to database, and display search results
 - Vlad + Natalia + Florian – pls merge ur stuff together so they work together *seamlessly*
 - Natasja pls logout btn
 - Presentation for midterm thingy
 - Introduction + Problem analysis
 - ♣ Florian
 - Requirements + Design (Figma)
 - ♣ Vlad
 - System architecture + Tech stack
 - ♣ Me
 - Until next meeting
 - ♣ Favourites front end
 - Oskar
 - ♣ Honestly, what functionality are we missing?
 - ♣ Recommendation based on user data as well
 - Bea
 - ♣ Spell checking / auto correction
 - Bea
 - ♣ Let's start writing the report for now
 - Introduction
 - Florian

- Methodology
 - Natalia
- Problem Analysis
 - Vlad
- Requirements
 - Natasja
- Design
 - Bea
- Implementation
 - Oskar

Week: 15

Present: Bea, Natalia, Natasja, Oskar, Vlad

Missing: Florian

- Agenda:
 - During meeting
 - ♣ What did they say during the presentation? Where else can we implement AI in our application?
 - What other functionalities do they want?
 - ♣ I want to check out how our application is doing now – haven't tried the filtration + sortation together, nor seen the results, nor the favouriting stuff
 - ♣ Autocorrect is useless if we cannot search (meaning we can hit enter in search bar and can view the results in the search page)
 - ♣ Meeting with Jukka 2025/04/24 13:30, and our weekly meeting immediately afterwards
 - ♣ Previous work – did everyone do it?
 - Favourites front end
 - Oskar
 - Recommendation based on user data as well
 - Bea
 - Spell checking / auto correction
 - Bea
 - Let's start writing the report for now
 - Introduction
 - ♣ Florian

- o Methodology
 - ♣ Natalia
 - o Problem Analysis
 - ♣ Vlad
 - o Requirements
 - ♣ Natasja
 - o Design
 - ♣ Bea
 - o Implementation
 - ♣ Oskar
- o Until next meeting
 - ♣ Florian, Natalia, Vlad
 - Everything on the main branch, first of all 😊
 - Book results displayed in the search page
 - Sortation intakes results from filtration (in back end)
 - Everything works together **Seamlessly**
 - Put sortation and filtration into separate files and have their own parameterized functions 😊
 - ♣ Bea
 - Recommendation part for sortation
 - Report: why we didnt fulfill the project requirements
 - ♣ Oskar
 - Finish implementation
 - ♣ Create test scripts ("navigation" and expected results)
 - Natasja

Week: 17

Present: Bea, Florian, Natalia, Natasja, Oskar

Missing: Vlad

- Agenda:
 - o During meeting
 - ♣ Previous work
 - Florian, Natalia, Vlad
 - o Everything on the main branch, first of all 😊
 - o Book results displayed in the search page
 - o Sortation intakes results from filtration (in back end)

- o Everything works together ****Seamlessly****
 - o Put sortation and filtration into separate files and have their own parameterized functions 😊
- Bea
 - o Recommendation part for sortation
 - o Report: why we didn't fulfil the project requirements
- Oskar
 - o Finish implementation
- Create test scripts ("navigation" and expected results)
 - o Natasja
- o Until next meeting
 - ♣ Report continued?
 - ♣ Componentize sortation
 - Bea
 - ♣ Too many book results displayed -> laggy page for like 10 seconds
 - Bea
 - ♣ Links not working for book results (literally a 10-minute fix max)
 - Bea
 - ♣ Scrollbar hidden behind filtration – doesn't matter much, just annoying and perhaps easily fixable
 - Bea
 - ♣ Recommendations take way too long, especially for account page and most definitely the search page
 - Account page
 - o Natalia
 - Search page
 - o Natasja
 - ♣ Filters bar not scrollable
 - Bea
 - ♣ Complete search bar, making it user friendly add all 10000 entries
 - Florian
 - ♣ Oskar
 - Finish implementation
 - ♣ What else?
 - ♣ Jukka's recommendations?

Present: Everyone :D

Missing:

- Agenda:
 - During meeting
 - ♣ Previous work
 - Componentize sortation
 - Bea
 - Too many book results displayed -> laggy page for like 10 seconds
 - Bea
 - Links not working for book results (literally a 10-minute fix max)
 - Bea
 - Scrollbar hidden behind filtration – doesn't matter much, just annoying and perhaps easily fixable
 - Bea
 - Recommendations take way too long, especially for account page and most definitely the search page
 - Account page
 - ♣ Natalia
 - Search page
 - ♣ Natasja
 - Filters bar not scrollable
 - Bea
 - Complete search bar, making it user friendly add all 10000 entries
 - Florian
 - Also, what else did you do to the search bar? I forgot
 - Oskar
 - Finish implementation
 - Until next meeting
 - ♣ WRITE YOUR REPORT PARTS, EVERYONE
 - ♣ Release date filtration (its nonexistent rn)
 - Nope
 - ♣ Alphabetical sort not working (for some reason)
 - Bea (just alpha instead of alphabetical)
 - ♣ Fix styling for homepage book pics
 - Oskar

- ♣ Recommendation for account page randomly pick 1k books and recommend from those
 - Natasja
- ♣ Recommendation for search page randomize 1k books if the filtered list is >1k and recommend from those
 - Natasja
- ♣ Report continued
 - Let's check what's needed
- ♣ Diagrams for report
 - Architecture
 - Natasja
 - Flow chart for recommendation (maybe multiple for all different types of recommendation we have?)
 - Natasja
 - Database
 - Oskar (including django account stuff)
- ♣ Componentize with interfaces so its kinda legal? Instead of just moving them to separate files
 - Natalia
- ♣ Finish detailed tests (Bea wrote steps and data in Other Stuff in Test Scripts 2 or sth)
 - Bea
- ♣ Execute the tests and find out what is failing ☹️
 - Vlad
- ♣ Florian, some user experience?
- ♣ Optimizing search bar (adding rules for matching words, displace the correction thingy)
 - Florian
- ♣ Any more features?

Week: 19

Present: Everyone :D

Missing:

- Agenda:
 - During meeting
 - ♣ Previous work

- WRITE YOUR REPORT PARTS, EVERYONE
- Fix styling for homepage book pics
 - Oskar
- Finish implementation of favourite books
 - Oskar
- Recommendation for account page randomly pick 1k books and recommend from those
 - Natasja
- Recommendation for search page randomize 1k books if the filtered list is >1k and recommend from those
 - Natasja
- Diagrams for report
 - Architecture
 - ♣ Natasja
 - Flow chart for recommendation (maybe multiple for all different types of recommendation we have?)
 - ♣ Natasja
 - Database
 - ♣ Oskar (including django account stuff)
- Componentize with interfaces so its kinda legal? Instead of just moving them to separate files
 - Natalia
- Finish detailed tests (Bea wrote steps and data in Other Stuff in Test Scripts 2 or sth)
 - Bea
- Alphabetical sort not working (for some reason)
 - Bea (just alpha instead of alphabetical)
- Execute the tests and find out what is failing 😊
 - Vlad
- Optimizing search bar (adding rules for matching words, displace the correction thingy)
 - Florian
- Until next meeting
 - ♣ Bea
 - Correct book page recommendation, so it isnt top 500 that share tags, but random 1k from 10k books
 - Add best rated to the algorithm, just like popularity
 - And change all this in the presentation lol or don't and only after it is implemented?
 - Edit plantuml diagrams for recommendation
 - Add the interfaces to recommender

- Ask krzysusu about uml diagrams
- Recommendation algorithm in himepage doesnt use search history??
- ♣ Prepare presentation for friday BY TODAY evening 😊
 - Content should be decided during the meeting, slides individually made, BY TODAY evening 😊
 - 1 min of introduction - Florian
 - 30 tech stack - Oskar
 - 1 min system architecture & component diagram - Natasja
 - 1.5 min search bar - Florian
 - 1.5 min recommendation - Bea
 - 30 seconds testing strategies - Natalia
 - 3 mins demo - Vlad
- ♣ Finish the report already 😊 lots of implementation is missing
- ♣ Add uml diagrams to report
- ♣ Finish report until Sunday!!!!
- ♣ Conclusion in Report
 - Natalia
- ♣ Favourite button text should be changed when toggled
 - Oskar
- ♣ What other features are missing??? What is still in progress???
What failed in testing???

Week: 20

Present: Everyone :D

Missing:

- Agenda:
 - During meeting
 - ♣ Previous work
 - Bea
 - Correct book page recommendation, so it isnt top 500 that share tags, but random 1k from 10k books
 - Add best rated to the algorithm, just like popularity
 - And change all this in the presentation lol or don't and only after it is implemented?

- o Edit plantuml diagrams for recommendation
 - o Add the interfaces to recommender
 - o Ask krzysusu about uml diagrams
 - o Recommendation algorithm in homepage doesn't use search history??
 - o ALSO
 - ♣ I did entire automation testing system + unit test cases + integration test cases
- Florian
 - o Added any test cases for search stuff?
- Finish the report already 😊 lots of implementation is missing
- Add uml diagrams to report
- Finish report until Sunday!!!!
- Conclusion in Report
 - o Natalia
- Favourite button text should be changed when toggled
 - o Oskar
- o Until next meeting
 - ♣ Test Florians code
 - Spelling correction
 - Tab event for filling in with suggestions
 - Oskar
 - ♣ Add more tests
 - Bea
 - ♣ Write testing part in report
 - Bea
 - ♣ Update diagrams
 - Natasja
 - ♣ Report done
 - Florian PLSSS by tonight
 - ♣ What's missing?
 - ♣ What components are still in development / need improvement?
 - ♣ Are all tests done?
 - ♣ All tests passing?
 - ♣ Improve front end
 - Vlad and Natalia

Week: 21

Present: Everyone :D

Missing:

- Agenda:
 - During meeting
 - ♣ Previous work
 - Test Florians code
 - Spelling correction
 - Tab event for filling in with suggestions
 - Oskar
 - Add more tests (performance and reliability)
 - Bea
 - Write testing part in report
 - Bea
 - Update diagrams
 - Natasja
 - Report done
 - Florian PLSSS by tonight
 - Improve front end
 - Vlad and Natalia
 - Until next meeting
 - ♣ Is the **code** completed finally?
 - ♣ Do all tests pass?
 - ♣ What is STILL NOT done? >:c
 - ♣ Who will fix what in the report according to Krzysztof's comments? - deadline is tomorrow midnight 😊 THURSDAY MIDNIGHT
 - Bea
 - Testing
 - Requirements
 - Abstract
 - Vlad
 - Introduction
 - Natalia
 - Problem Analysis
 - Florian
 - Please fix so page numbering starts at introduction (should it not? ldk)
 - Remove footnotes
 - Test scripts
 - Meeting logs

- Natasja
 - Meeting log as 1 pdf
 - Censorship
- Oskar
 - Deployment diagram
 - Interaction diagram
- ♣ Have we all read the report? If not, read it by FRIDAY EIGHT
Post meridiem 8pm 😊 This is NOT optional, I expect everyone
to read every single character of this report, and point out
mistakes 😊 Because we all know there are mistakes
 - Add comments in DISCORD